烟幕干扰弹的投放策略

摘　要

本文针对无人机投放烟幕干扰弹对来袭导弹进行干扰的问题，研究在不同投放策略下实现最长有效遮蔽时间的最优方案。

针对问题一，基于固定策略计算烟幕弹对导弹 M1 的有效遮蔽时长；通过把导弹当作一个发光源，光源通过烟幕弹形成了一个圆锥形的视野盲区，这个圆锥是一个关于时间t的函数；当圆锥在 z=0 这个平面上的投影椭圆完全覆盖真目标圆柱在 z=0 的投影圆，同时圆锥在 z=10 上也满足对应条件，烟幕弹此时完全遮蔽了真目标，此时记为时间t_1，遮蔽时长的上限由导弹超越烟幕弹的时刻t_2所决定，因此最终的有效遮蔽时长为$t_2 - t_1 = 1.4244s$。

之后的问题二，三，四，五的解决均把真目标当作一个坐标为$(0,200,5)$的质点P。

针对问题二，求解无人机的投弹策略，最大化遮蔽时长，首先求出导弹和烟幕弹的位置函数$M(t), S(t)$，然后定义了值域为[0,1]的遮蔽质量函数$Q(t)$，将向量$\overrightarrow{MP}$同导弹视野盲区圆锥的中轴线向量$\overrightarrow{MS}$的夹角定义为β，圆锥的半顶角记为α，β越接近于α，$Q(t)$的值越接近于1，表明遮盖效果越好。这样在题目给定的约束条件下，通过最大化$Q(t)$在$\left[t_{爆炸}, t_{爆炸}+20\right]$区间内关于时间$t$的积分，可以得到最大的有效遮蔽时长；由于$Q(t)$可微，因此采用网格搜索提供初始值+梯度下降优化算法联合求解。最终求得的最大有效遮蔽时间约为 4.08s，无人机飞行速度为 133.64m/s，航向角为 7.41 度。

针对问题三，同样延续问题一二的思想，求解无人机 FY1 投 3 枚烟幕弹的策略，与问题二不同的是，遮蔽质量函数$Q(t)$的参数空间包含了 8 个变量，增加了任意两个烟幕弹的投放时间大于 1s 的约束条件。同样是在给定的约束条件下，通过最大化三个烟幕弹$Q(t)$在$\left[t_{爆炸}, t_{爆炸}+20\right]$区间内关于时间$t$的积分之和，得到最大的有效遮蔽时长。采用差分进化的暴力搜索算法，得到最大的有效遮蔽时长为 6.2503s。

针对问题四，要求利用 3 架无人机各自投放一枚烟幕弹干扰导弹 M1，需要考虑多机协同的问题，在前三问的基础上构建每架无人机投放起爆形成的烟幕中心和导弹 M1 的运动学模型，再构建遮蔽质量函数量化遮蔽效果，以多烟幕遮蔽质量最大值的积分为目标函数，结合无人机参数约束，用“差分进化算法+局部精细化”优化 12 维决策变量。最终得最大有效遮蔽时间 12.336s，三机分别贡献 3.07s、6.40s、3.96s 的遮蔽时间，实现协同遮蔽。

针对问题五，求解五架无人机，每架无人机至多投放 3 枚烟幕弹，实施对 M1,M2,M3 的干扰的投放策略。在问题二至四的模型基础上，进一步考虑多智能体协同优化与高维参数空间搜索, 采取分阶段优化策略：先固定无人机飞行参数，优化烟幕弹投放与起爆时间；再以所得结果为初始值，进行全参数优化。通过差分进化算法求解，最终求得总的遮蔽时间为 23.63s。

模型的优点是使用了遮蔽质量函数$Q(t)$，一是可以把真目标是否被遮蔽的离散条件转化成连续函数，方便后续的优化；二是提供了一个修正系数$maxQ(t)$，用它乘以原始有效遮蔽区间，减小由于把真目标当作质点所带来的误差。

模型的缺点一是除问题一外均将真目标简化为质点，会导致遮蔽效果计算存在偏差；二是采用的差分进化算法在参数设置上存在一定主观性，且面对高维参数空间时，搜索效率和寻优精度仍有提升空间。

关键词：遮蔽质量函数，网格搜索，梯度下降，差分进化

一、问题重述

1.1 问题背景

烟幕干扰弹可形成球状遮蔽云团，用于在来袭导弹和真目标之间形成视野遮蔽盲区。现用无人机定点精准投放，且每架无人机投放两枚烟幕干扰弹至少间隔1s；烟幕干扰弹脱离无人机后只受重力运动，起爆后形成球状云团以 3m/s 匀速下沉，云团中心 10m 范围内 20s 内可有效提供遮蔽。假目标、真目标、导弹 M1-M3、无人机 FY1-FY5 的物理信息已知。题目要求设计投放策略（含方向、速度、投弹点、起爆点），最大化对真目标的有效遮蔽时间。

1.2 问题回顾

问题 1：已知无人机 FY1 以 120m/s 朝假目标飞行、受领任务 1.5s 后投放 1 枚烟幕弹且间隔 3.6s 起爆，求对导弹 M1 的有效遮蔽时长。

问题 2：针对无人机 FY1 投放 1 枚烟幕弹干扰 M1 ，确定飞行方向、速度、投放点及起爆点以最大化遮蔽时间。

问题 3 规划无人机 FY1 投放 3 枚烟幕弹干扰 M1 的策略。

问题 4：制定 FY1、FY2、FY3 三架无人机各投 1 枚烟幕弹干扰 M1 的策略。

问题 5 设计 5 架无人机（每架最多投 3 枚）干扰 M1、M2、M3 三枚来袭导弹的烟幕弹投放策略。

二、模型假设

为了方便模型的建立与模型的可行性，本部分首先提出一些假设，使得模型更加完备，预测的结果更加合理。

1. 题目给出的所有数据真实有效。
2. 假设烟幕弹脱离无人机后的运动仅受重力的影响，烟幕云团起爆后瞬时成 10m 半径标准球体。
3. 无人机在收到命令后的瞬时调向耗时为$0s$。
4. 除第一问外，假设将真目标视为一个质点。
5. 无极端天气、电磁干扰等外部因素，各物体运动均按理想规律进行，无随机误差。

三、符号说明

为方便模型的建立与求解，本部分对使用到的关键全局符号进行以下说明，注意本文各小题的局部变量符号相互独立，范围仅限于每个问题小节。

符号	符号说明	值/单位
g	重力加速度	$9.80665\mathrm{m/s^2}$
V_{Fi}	无人机FYi的飞行速度大小	$70\sim140m/s$

V_M	导弹飞行速度大小	$300m/s$
V_s	烟幕云团下沉速度大小	$3m/s$
t_{Fi}	无人机FYi投弹前的飞行时间	s
t_s	烟幕弹从投放到起爆的时间	s
r_S	烟幕球状云团的半径	$10m$
M_i	导弹M_i的初始坐标位置	(x_{Mi}, y_{Mi}, z_{Mi})
F_i	无人机FYi的初始坐标位置	(x_{Fi}, y_{Fi}, z_{Fi})
S_1	烟幕起爆时云团中心S_1的位置	(x_{S1}, y_{S1}, z_{S1})
M_i'	烟幕起爆时导弹M_i的位置	$(x_{Mi}', y_{Mi}', z_{Mi}')$
S_t	烟幕起爆后t时刻烟幕中心的位置	(x_S, y_S, z_S)
M_{it}	烟幕起爆后t时刻导弹M_i的位置	$(x_{Mit}, y_{Mit}, z_{Mit})$
P	真目标下底面的圆心位置	(0,200,0)
O	假目标的坐标,空间坐标系的原点	(0,0,0)

四、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 模型的建立

步骤 1：烟幕中心与导弹的运动分析。

首先计算烟幕弹爆开的中心位置，烟幕弹先随无人机做匀速直线运动，然后被无人机抛射后做平抛运动，起爆后做匀速直线运动，且该运动过程中烟幕弹中心始终在xoz平面内，因此烟幕弹起爆时烟幕中心$S_1 = (x_{S1}, y_{S1}, z_{S1})$的坐标位置：

$$\begin{cases} x_{S1} = x_{F1} - (t_{F1} + t_s) \cdot V_{F1} \\ y_{S1} = 0 \\ z_{S1} = z_{F1} - \frac{1}{2} \cdot g \cdot {t_s}^2 \end{cases} \tag{1}$$

然后计算烟幕起爆时导弹M_1的位置，导弹始终做匀速直线运动，方向指向原点，先计算该导弹在被发现到烟幕弹起爆这段时间飞行的总路程：

$$S_{M1} = V_M \cdot (t_{F1} + t_S) \tag{2}$$

设导弹指向原点的单位向量为$\vec{e} = (\cos\theta, 0, \sin\theta)$，其中：

$$cos\theta = \frac{x_{M1}}{\sqrt{{x_{M1}}^2 + {z_{M1}}^2}} \tag{3}$$

$$\sin\theta = \frac{z_{M1}}{\sqrt{{x_{M1}}^2 + {z_{M1}}^2}} \tag{4}$$

烟幕弹起爆时刻导弹$M_1' = (x_{M1}', y_{M1}', z_{M1}')$的坐标位置：

$$\begin{cases} {x_{M1}}' = x_{M1} - S_{M1} \cdot \cos\theta \\ {y_{M1}}' = 0 \\ {z_{M1}}' = z_{M1} - S_{M1} \cdot \sin\theta \end{cases} \tag{5}$$

同时导弹的速度分量为：

$$\begin{cases} V_{M1x} = V_M \cdot \cos\theta \\ V_{M1y} = 0 \\ V_{M1z} = V_M \cdot \sin\theta \end{cases} \tag{6}$$

烟幕起爆后的 t 时刻，烟幕中心位置$\overrightarrow{\boldsymbol{S_t}} = (x_S, y_S, z_S)$,导弹$M_1$的位置$\overrightarrow{\boldsymbol{M_{1t}}} = (x_{M1t}, y_{M1t}, z_{M1t})$，且二者均做匀速直线运动。

烟幕弹的动态实时位置坐标：

$$\begin{cases} x_S = x_{S1} \\ y_S = 0 \\ z_S = z_{S1} - V_s \cdot t \end{cases} \tag{7}$$

导弹的动态实时位置坐标：

$$\begin{cases} x_{M1t} = x'_{M1} - V_{M1x} \cdot t \\ y_{M1t} = 0 \\ z_{M1t} = z'_{M1} - V_{M1z} \cdot t \end{cases} \tag{8}$$

步骤 2：推导导弹视野盲区圆锥体被$z = 0$平面所截取的椭圆的方程。

将导弹当作一个发光源，这个发光源被烟幕弹遮掩后，会形成一个以导弹瞬时位置为顶点、烟幕球为内切球的圆锥。烟幕起爆后 t 时刻，圆锥盲区的轴线方向即为烟幕中心与导弹的位置向量差：

$$\vec{\boldsymbol{d}} = \overrightarrow{\boldsymbol{S_t}} - \overrightarrow{\boldsymbol{M_{1t}}} \tag{9}$$

其模长为：

$$L = \| \vec{\boldsymbol{d}} \| \tag{10}$$

导弹到目标点与烟幕边界的切线长度：

$$M_{1t}T = \sqrt{L^2 - r_S^2} \tag{11}$$

临界夹角的余弦值：

$$\cos\theta = \frac{M_{1t}T}{L} = \frac{\sqrt{L^2 - r_S^2}}{L} \tag{12}$$

这个临界夹角即为该圆锥的半顶角。锥体表面任取一点$\vec{\boldsymbol{Q}} = (x, y, z)$与导弹位置的向量为：

$$\overrightarrow{\boldsymbol{M_{1t}Q}} = \vec{\boldsymbol{Q}} - \overrightarrow{\boldsymbol{M_{1t}}} \tag{13}$$

圆锥面方程即为：

$$\frac{\overrightarrow{\boldsymbol{M_{1t}Q}} \cdot \vec{\boldsymbol{d}}}{\| \overrightarrow{\boldsymbol{M_{1t}Q}} \| \cdot \| \vec{\boldsymbol{d}} \|} - \cos\theta = 0 \tag{14}$$

为了确保保护目标处于导弹视野盲区内，将圆锥面方程与$z = 0$平面联立，求出交界椭圆方程。在$z = 0$平面上任意一点$N = (x, y, 0)$,导弹与该点的向量：

$$\overrightarrow{\boldsymbol{M_{1t}N}} = (x - x_{M1t}, y - y_{M1t}, -z_{M1t}) \tag{15}$$

遮蔽视野盲区在$z = 0$平面上应满足：

$$\frac{\overrightarrow{\boldsymbol{M_{1t}N}}\cdot\vec{\boldsymbol{d}}}{\|\overrightarrow{\boldsymbol{M_{1t}N}}\|\cdot\|\vec{\boldsymbol{d}}\|}=\cos\theta \tag{16}$$

带入数值并整理可得到一个包含时间参数 t 的二次曲线方程，这个椭圆即为导弹的视野盲区在平面$z=0$上面的投影。

步骤 3：计算遮蔽的临界时间。

有效遮蔽要求将真目标完全覆盖在导弹视野盲区内，即要求圆锥在$z=0$和$z=10$这两个平面上的投影椭圆分别将真目标圆柱体在这两个平面的投影圆完全覆盖，经过计算发现导弹会在t_2时刻穿过烟幕球体中心，两个投影椭圆截面随着导弹及烟幕弹的运动会向x轴正方向移动，遮蔽盲区会先覆盖目标的顶端部分，因此只需要判断当$z=0$平面的投影椭圆将目标圆柱体的底面圆相切时，$z=10$平面的投影椭圆是否也将同平面内的圆柱上底面圆覆盖，就可以判定此时刻是否为完全遮蔽开始的临界时间。

因此将$z=0$平面上椭圆方程与真目标所在位置的圆方程联立，计算出椭圆与圆的相切时刻t_1。

平面$z=0$内圆的方程为：

$$x^2+(y-200)^2=7^2 \tag{17}$$

如果此时刻$z=10$平面的投影椭圆是否也将的圆柱上底面圆覆盖，则这两个时刻之差即为有效遮蔽时间t_v。

$$t_v=t_2-t_1 \tag{18}$$

如果未能覆盖，则说明当真目标底部刚好进入视野盲区的时，该目标圆柱体的顶端重新暴露在导弹视野范围内，烟幕弹根本无法形成有效遮蔽，有效遮蔽时长为0s。

5.1.2 问题一结果

如图 1，$t=2.929$s时导弹视野盲区在$z=10$的投影椭圆与目标圆柱在$z=0$平面的投影圆相切，此时，如图 2 所示经过计算发现导弹视野盲区在$z=0$平面上的投影椭圆能够完全覆盖同平面内的圆柱上底面圆。

经计算观察，在投影椭圆穿过真目标之前，导弹会率先超越烟幕，因此以导弹超越烟幕的时间作为遮蔽时长的上界。导弹穿过烟幕中心的时刻$t_2=4.3534$，有效遮蔽时长 $t_v=t_2-t_1$=1.4244s。

因此问题 1 的模型求解结果是 $1.4244s$。

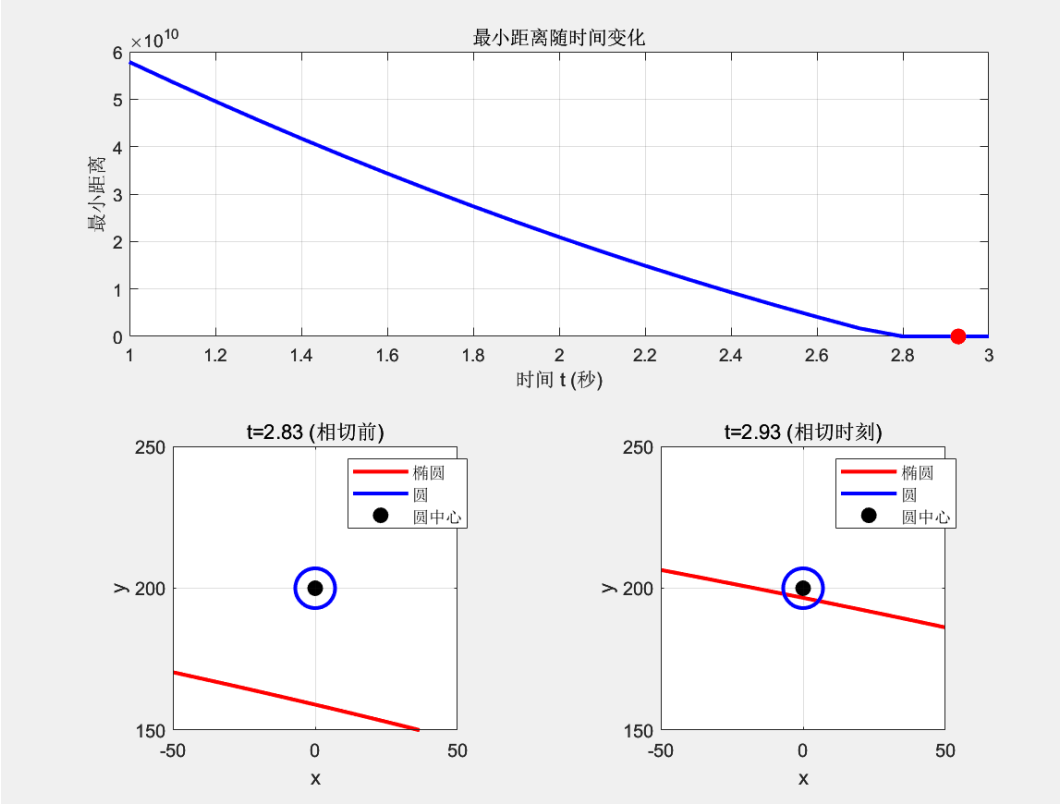


图 1 烟幕投影椭圆与目标投影圆的关系图



图 2 当 z=0 平面烟幕椭圆与目标下底圆相切时，z=10 平面上烟幕投影椭圆与目标上底圆的位置关系

5.2 问题二模型的建立与求解

5.2.1 模型的建立

步骤 1：确定烟幕中心与导弹的运动方程。

设无人机$FY1$的飞行方向、飞行速度大小、烟幕干扰弹随无人机运动的时间段（匀速直线运动）、烟幕干扰弹从被抛出到被引爆的时间段（平抛运动）分别为θ、V_1、t_1、t_2，其中θ这个角度是从 x 轴正向开始计算，逆时针方向为正取值0~360度。参照问题一的运动分析背景，给定这四个参数后就可以确定出烟幕弹起爆之后的 t 时刻导弹和烟幕云团中心的位置函数。

设无人机飞行方向的单位向量$\vec{f} = (\cos\theta, \sin\theta, 0)$，烟幕弹中心的位置变化向量为$\vec{\Delta}$,则：

$$\vec{\Delta} = V_1 \cdot (t_1 + t_2) \cdot \vec{f} + \left(0,0,-\frac{1}{2}g{t_2}^2\right) \tag{19}$$

t时刻烟幕中心的实时位置向量：

$$\vec{S} = \overrightarrow{F_1 O} + \vec{\Delta} \tag{20}$$

设导弹指向原点的单位向量为$\vec{e} = (\cos\gamma, 0, \sin\gamma)$，其中：

$$cos\gamma = \frac{x_{M1}}{\sqrt{{x_{M1}}^2 + {z_{M1}}^2}} \tag{21}$$

$$\sin\gamma = \frac{z_{M1}}{\sqrt{{x_{M1}}^2 + {z_{M1}}^2}} \tag{22}$$

t时刻导弹的实时位置向量：

$$\vec{M} = \overrightarrow{\mathrm{M_1O}} - \mathrm{V_M} \cdot (\mathrm{t_1} + \mathrm{t_2}) \cdot \vec{\mathrm{e}} - \mathrm{V_M} \cdot \mathrm{t} \cdot \vec{\mathrm{e}} \tag{23}$$

步骤 2：构建平滑变化的遮蔽质量函数，使真目标是否被遮蔽的离散条件转化成连续函数。

延续问题一的建模思想，把导弹M_1想象成一个光源，其被烟幕云团S遮挡的视野盲区是一个圆锥，真目标设为点P, 设圆锥的半顶角α，则：

$$cos\alpha = \sqrt{\left(1 - \left(\frac{r_s}{|\overrightarrow{MS}|}\right)^2\right)} \tag{24}$$

定义$M1$同真目标P的向量$\overrightarrow{\boldsymbol{MP}}$同圆锥的中轴线$\overrightarrow{MS}$的夹角为β ，则：

$$cos\beta = \frac{\overrightarrow{|MP} \cdot \overrightarrow{MS}|}{|\overrightarrow{MP}||\overrightarrow{MS}|} \tag{25}$$

定义t时刻烟幕云团对真目标的遮蔽质量函数$Q(t)$:

若 $cos\beta \geq cos\alpha$ 即真目标在烟幕的遮蔽角内，则:

$$Q(t) = \sqrt{\left(\frac{cos\beta - cos\alpha}{1 - cos\alpha}\right)} \tag{26}$$

若 $\cos\beta < \cos\alpha$ 即真目标在烟幕的遮蔽范围之外，则:

$$Q(t) = \left(1 - \frac{cos\alpha - cos\beta}{cos\alpha + 1}\right)^3 \tag{27}$$

即为：

$$Q(t) = \begin{cases} \sqrt{\left(\dfrac{cos\beta - cos\alpha}{1 - cos\alpha}\right)}, & cos\beta \geq cos\alpha \\ \left(1 - \dfrac{cos\alpha - cos\beta}{cos\alpha + 1}\right)^3, & \cos\beta < \cos\alpha \end{cases} \tag{28}$$

这样，函数$Q(t)$通过平方根和三次方这种平滑过渡形式，避免了离散条件下的突变，使遮蔽质量在 0 到 1 之间取得连续值，为后续的梯度优化算法提供基础。

步骤 3：确定优化目标函数。

算法的目标是最大化烟幕云团对目标的总遮蔽效果，即遮蔽质量在有效时间内的积分：

$$J(\theta)=\int_{t_s}^{t_s+20} Q(t)dt \tag{29}$$

积分下限是烟幕云团的爆炸时间$t_s=(t_1+t_2)$，积分上限是 t_s+20s，20s 即是烟幕持续时长。

则问题二转化为如下的优化问题：

$$min_\theta -J(\theta) \tag{30}$$

$$s.t.\begin{cases}70 \le v \le 140\\ 0 \le \theta \le 360\\ 0 \le t_1 \le 10\\ 0 \le t_2 \le 20\end{cases} \tag{31}$$

步骤 4:用网格搜索 + 梯度下降的方法联合优化目标。

由于梯度下降算法对初值较为敏感，因此选择网格搜索初始值加梯度下降优化的算法进行优化。首先在参数空间内生成网格初始点，然后对每个初始点使用拟牛顿法求解局部最优值，最后得到结果。

步骤 5：把遮蔽质量函数关于时间 t 的积分转化为有效遮蔽时长：

$$t_{有效}=\int_{t_{min}}^{t_{max}} max\,Q(\mathrm{t})dt$$

上述公式的含义是把$Q(t)$的最大值(一个 0 到 1 之间的数)代替$Q(t)$，乘以遮蔽时间。由于一开始模型为了简便，把真目标简化成了一个质点，实际上当这个质点在导弹的视野盲区范围内时，真目标的边缘可能暴露在了导弹视线内，真实的有效遮蔽时间就不会是单纯的遮蔽时间，而是乘了一个系数，这个系数就是$max\,Q(\mathrm{t})$，这样就提高了模型精准度。

5.2.2 问题二结果

经计算发现，无人机以 133.64m/s 的速度、7.47°航向角飞行时，在 0.0003s 投放烟幕弹并于 0.4111s 起爆，此时在坐标(17854.51,7.15,1799.17)处烟幕弹爆开，可形成 4.082 的最大遮蔽时间，表 2 为问题 2 的结果。

图 3 呈现了遮蔽质量随时间变化的曲线，能看到遮蔽质量先有小幅稳定阶段，随后快速上升至峰值，接着又迅速下降至几乎为零。图 4 对比了导弹和烟幕的高度随时间的变化情况，导弹高度呈持续下降趋势，烟幕高度则是缓慢下降。图 5 展示了导弹与烟幕的轨迹分布，清晰呈现出目标、爆开位置以及二者的轨迹走向。

表 1 问题 2 结果

参数	最优值
飞行方向（航向角θ） (°)	7.47
飞行速度V_1 (m/s)	133.64
烟幕投放点 X 坐标(m)	17800.04
烟幕投放点 Y 坐标(m)	0.0052
烟幕投放点 Z 坐标(m)	1800
烟幕起爆 X 坐标(m)	17854.51
烟幕起爆 Y 坐标(m)	7.15

烟幕起爆 Z 坐标(m)	1799.17
最大有效遮蔽时间(s)	4.08
烟幕起爆时间(s)	0.41

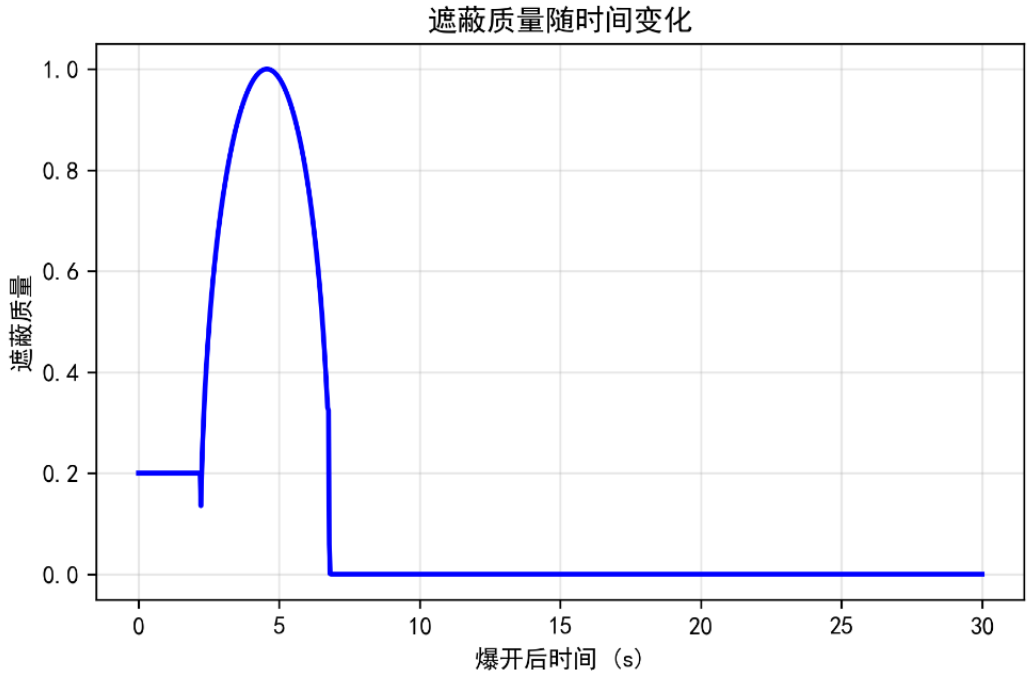


图 3 遮蔽质量函数随时间的变化曲线

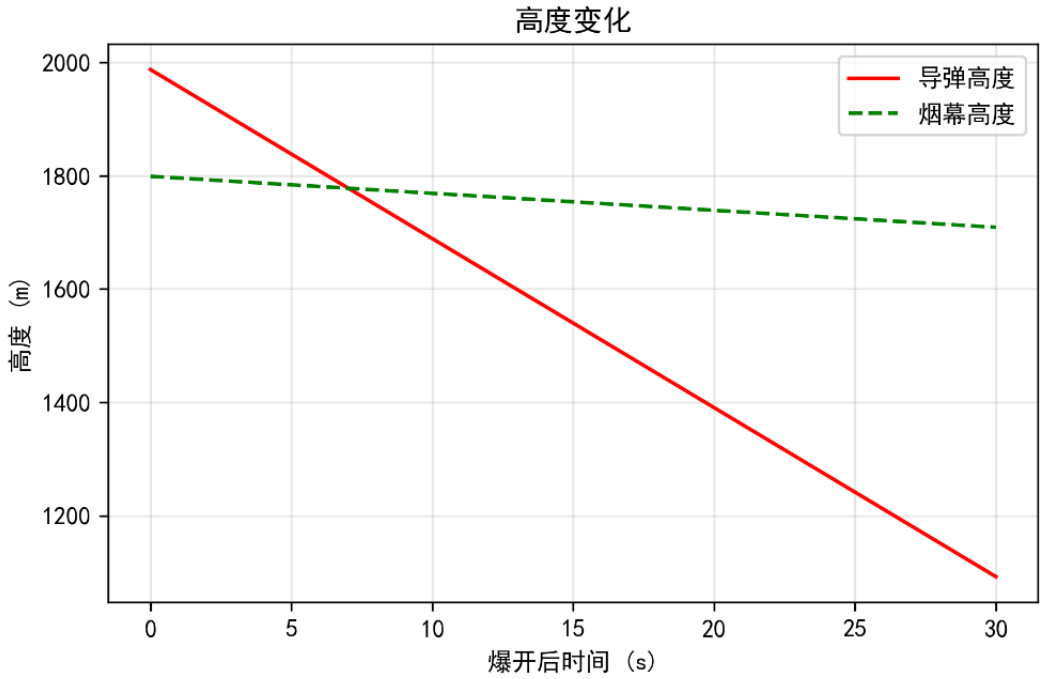


图 4 导弹与烟幕高度随时间变化对比图

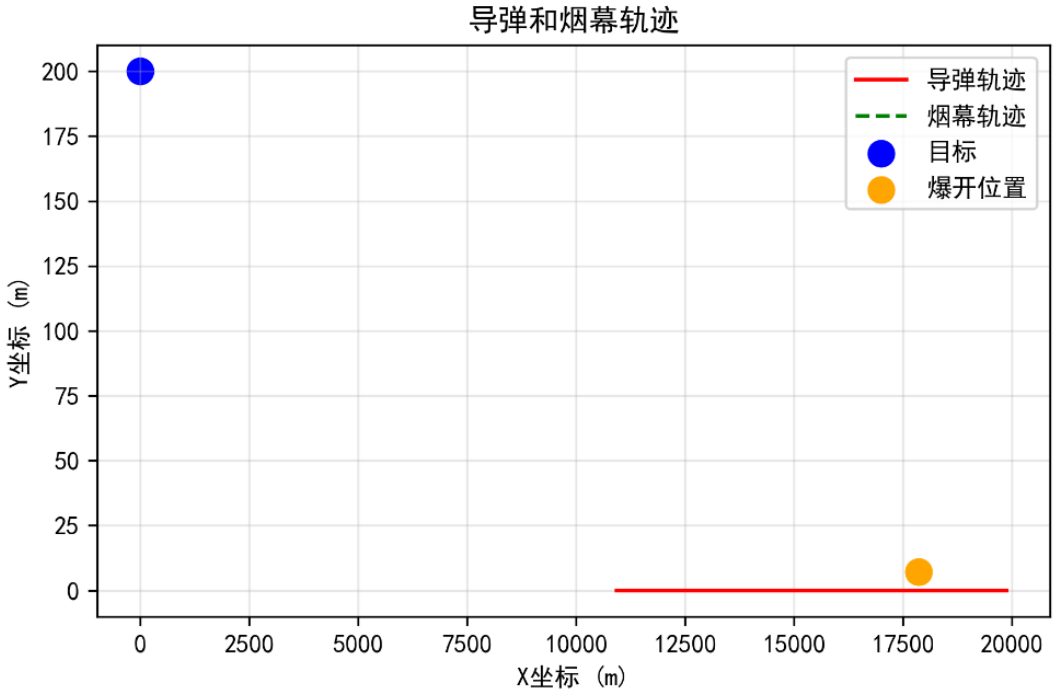


图 5 导弹与烟幕轨迹分布示意图

5.3 问题三模型的建立与求解

5.3.1 模型的建立

问题三要求给出无人机 FY1 投放三枚烟幕弹的投放策略，相较于问题二，多了两枚烟幕弹的投放时间$t_{1,i}$和爆炸时间$t_{1,j}$，并且对于所有的$i \neq j$有相邻两个烟幕弹的投放时间大于 1s 的约束条件：

$$|t_{1,i} - t_{1,j}| \geq 1 \tag{32}$$

步骤一：参照问题二，求解三个烟幕中心$\boldsymbol{S_i(t)}$与导弹$\boldsymbol{M(t)}$的运动方程。此步骤省略。

步骤二：延续问题二化离散为连续的思想，求解总的遮蔽质量函数

参照问题二提出的遮蔽质量函数$Q(t)$，本题的目标函数应为所有烟幕弹在整个有效时间内提供的最大遮蔽质量的积分:

$$J = \int_{t_{min}}^{t_{max}} \max_i Q(\mathbf{M}(t), \mathbf{S}_i(t), \mathbf{T}_r) dt \tag{32}$$

积分的上限仍然是积分下限加上烟幕持续时间 20s。这样，问题三就转化为了如下优化问题：

决策变量:

$$x = [v_u, \theta, t_{1,1}, t_{2,1}, t_{1,2}, t_{2,2}, t_{1,3}, t_{2,3}] \tag{33}$$

目标函数:

$$min_x - J(x) \tag{34}$$

约束条件:

$$s.t.\begin{cases} 70 \le v \le 140 \\ 0 \le \theta \le 360 \\ 0 \le t_1 \le 10 \\ 0 \le t_2 \le 20 \\ |t_{1,i} - t_{1,j}| \ge 1 \end{cases} \tag{35}$$

步骤三：差分进化算法+梯度下降法组合优化

虽然目标函数仍然可导，但是传统的梯度下降法对于 8 个参数的目标函数来说效果不佳，因此选择使用差分进化算法(Differential Evolution)来进行暴力搜索，然后对搜索局部再使用梯度下降法进行精细优化。

5.3.2 问题三结果

设置差分进化算法的迭代次数为 100 次，经过梯度下降的优化后，最大的有效遮蔽时间收敛到 6.2503s，即下图曲线与 x 轴的面积。

图 6 展示了三枚烟幕弹综合遮蔽质量随时间的变化，能看到遮蔽质量先有小幅平稳阶段，之后快速上升并出现波动峰值，最后迅速下降至零，还标注了各烟幕弹的爆开时间。图 7 呈现了导弹和三枚烟幕弹在 XY 平面的轨迹，清晰显示出目标位置、各烟幕弹的爆开位置以及导弹和烟幕弹的轨迹走向，表示出此时三枚烟幕弹良好的遮蔽效果。

问题三的结果：

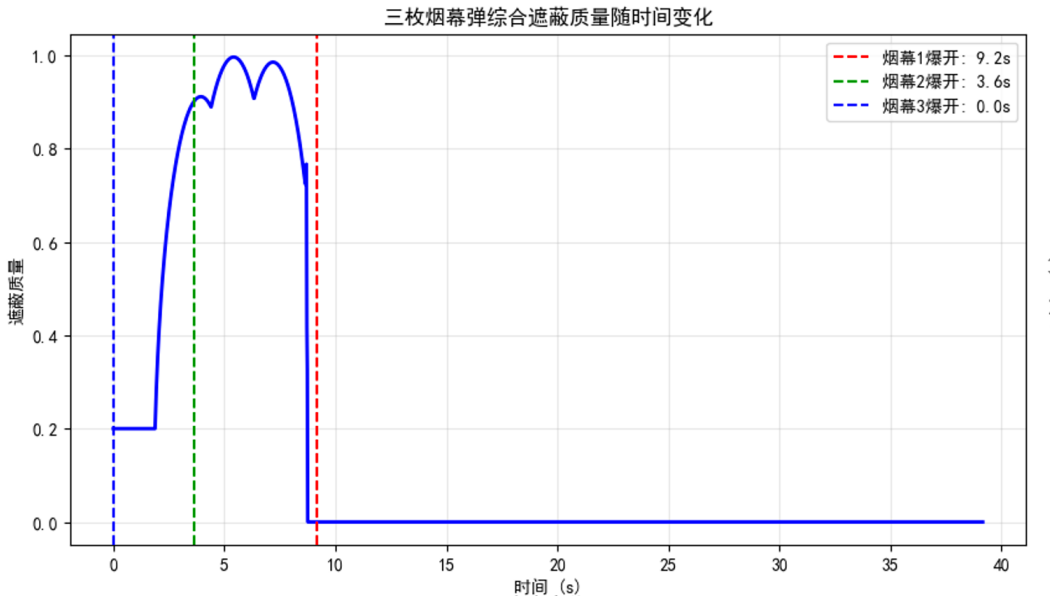


图 6 三枚烟幕弹综合遮蔽质量随时间的变化曲线

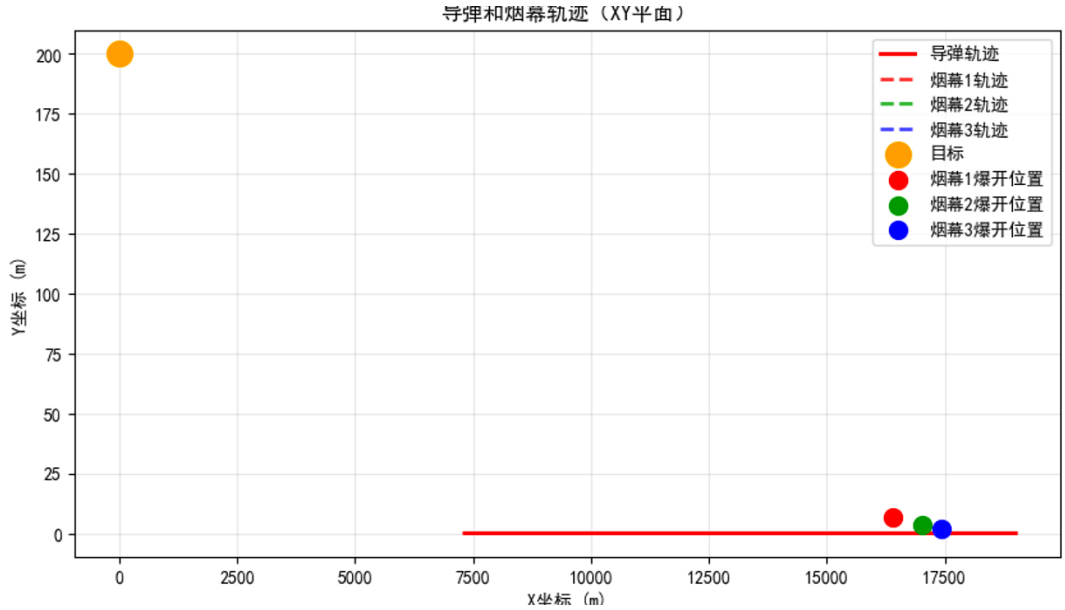


图 7 导弹和烟幕轨迹示意图（XY 平面）

5.4 问题四模型的建立与求解

问题四要求利用 3 架无人机各自投放一枚烟幕弹，相较于问题三，需要考虑多机协同的调度。本问题的解答建立在前三个问题的模型框架基础上，每架无人机FY_i的投放策略由四个变量决定，分别是无人机的飞行速度大小V_i、无人机的航向角θ_i、无人机从受领任务到投放烟幕的时间t_{1i}、烟幕弹从投放到起爆的时间t_{2i}决定。

5.4.1 模型的建立

步骤 1：参照问题二，求解每个烟幕中心$\boldsymbol{S_i(t)}$与导弹$\boldsymbol{M(t)}$的运动方程。此步骤省略。

步骤 2：确定遮蔽时间积分目标函数及约束条件。

问题四的决策变量是一个 12 维的向量，代表了三架无人机$(FY1, FY2, FY3)$各自的投放策略：

$$\vec{\boldsymbol{X}} = [V_1, \theta_1, t_{11}, t_{21}, V_2, \theta_2, t_{12}, t_{22}, V_3, \theta_3, t_{13}, t_{23}] \tag{36}$$

与问题二化离散为连续的思想一致，定义烟幕i在烟幕弹起爆后的t时刻对导弹M_1的遮蔽质量函数为$Q_i(t)$。$Q_i(t)$ 是第 i 枚烟幕弹在时刻 t 对目标点的遮蔽质量函数（取值0~1），其计算依赖于该时刻导弹位置 $\boldsymbol{M}(t)$ 和烟幕位置 $\boldsymbol{S_i}(t)$。

进一步地，将问题四的目标函数定义为三枚烟幕弹在整个仿真时间段内对导弹M_1造成的总遮蔽效果积分。其核心思想是：在任意时刻 t，导弹受到的遮蔽效果取当前所有已起爆且仍在有效期内的烟幕弹中遮蔽质量的最大值。

则目标函数$J(\mathbf{X})$为：

$$J(\mathbf{X}) = \int_{t_{\text{start}}}^{t_{\text{end}}} \max_{i \in \mathcal{A}(t)} Q_i(t) dt \tag{37}$$

其中，$t_{\text{start}}=0$是任务开始时刻，$t_{\text{end}}=80$是一个足够大的仿真结束时间，超过了导弹 M1 抵达原点的时间。$\mathcal{A}(t)$ 是时刻 t 下活跃烟幕弹的集合，活跃烟幕弹要求烟幕弹起爆并且处于有效持续时间内。

则问题四转化为如下的优化问题：

$$min_X -J(X) \tag{38}$$

$$s.t.\begin{cases} 70 \le V_i \le 140 \\ 0 \le \theta_i \le 360 \\ 0 \le t_{1i} \le 10 \\ 0 \le t_{2i} \le 20 \\ z_{S_i} \ge 0 \end{cases}, i=1,2,3 \tag{39}$$

步骤 3：通过差分进化优化算法。

问题 4 需要优化 3 架无人机的投放策略，总维度是 12 维，这种高维空间会导致传统的暴力搜索或简单贪心算法完全失效，同时优化目标是烟幕对真目标的总遮蔽积分，是一个有多个约束条件的非线性目标函数，可能存在多个局部最优解，要求算法具有全局寻优和跳出局部最优的能力，因此选择使用差分进化算法高效搜索最优组合。

问题四需要优化的参数维度是 12 个，首先根据问题情况设定好参数。为了保证种群多样性，覆盖足够多的投放策略组合，又不会因为种群过大而导致计算量激增，将种群大小$POPULATION_{SIZE}$设定为50。为了平衡搜索精度和计算效率，最大迭代次数$MAX_{GENERATIONS}$设定为1000，完全足够使算法从随机初始解逐步收敛到较优解，由于本题的无人机参数可行域较宽，需要一定的变异幅度来探索新的策略组合，因此将变异因子F_{DE}设定为0.8。同时，变异因子设定为0.8 又不会因变异过强导致种群波动过大，保证算法能稳定收敛到较优区域。本题 3 架无人机需要协同配合，高交叉概率能让算法更快地将 “FY1 的优质参数”“FY2 的优质参数” 等组合起来，形成更优的协同策略，为了加速融合不同候选解的优势，将高交叉概率CR_{DE}设定为0.9。设置好参数后进行 1000 代差分进化，确保结果收敛可靠。并在后期进一步通过$Nelder-Mead$单纯形法进行局部搜索，精细调节参数，提升解的质量。

5.4.2 问题四结果

目标函数经过差分进化优化和局部精细化优化后，最大遮蔽时间为 12.336s，三架无人机的具体策略如下表。

图 8 是 DE 算法适应度进化曲线，可看出经过大概 400 次迭代后能得到较为稳定且较好的结果。图 9 是导弹烟幕 XY 空间轨迹示意图，展示出导弹及各烟幕、真位置之间近乎共线，良好展现出烟幕的遮蔽效果。图 10 是三无人机遮蔽质量动态曲线，展现出三枚烟雾弹的遮蔽时段。图 11 是单无人机干扰时长柱状图，FY2（6.40 秒）最长，FY3（3.96 秒）次之，FY1（3.07 秒）最短。

问题四结果如表 2 所示，累计有效遮蔽总时长为12.336172 s。

表 2 问题 4 结果

无人机	FY1	FY2	FY3
飞行速度v_u(m/s)	139.669	140.000	140.000
飞行方向（航向角θ）(°)	0.000	265.378	111.323
投放时间 t1 (s)	0.100	3.195	10.000
起爆延迟 t2 (s)	0.100	6.445	11.423

投放 X 坐标(m)	17814.0	11964.0	5490.9
投放 Y 坐标(m)	-0.0	954.1	-1695.8
投放 Z 坐标(m)	1800.0	1400.0	700.0
烟幕爆开时间(s)	0.200	9.640	21.423
烟幕爆开 X 坐标(m)	17827.9	11891.3	4909.4
烟幕爆开 Y 坐标(m)	-0.0	54.8	-206.1
烟幕爆开 Z 坐标(m)	1800.0	1196.5	60.6
单独遮蔽贡献(s)	3.073949	6.399708	3.962478

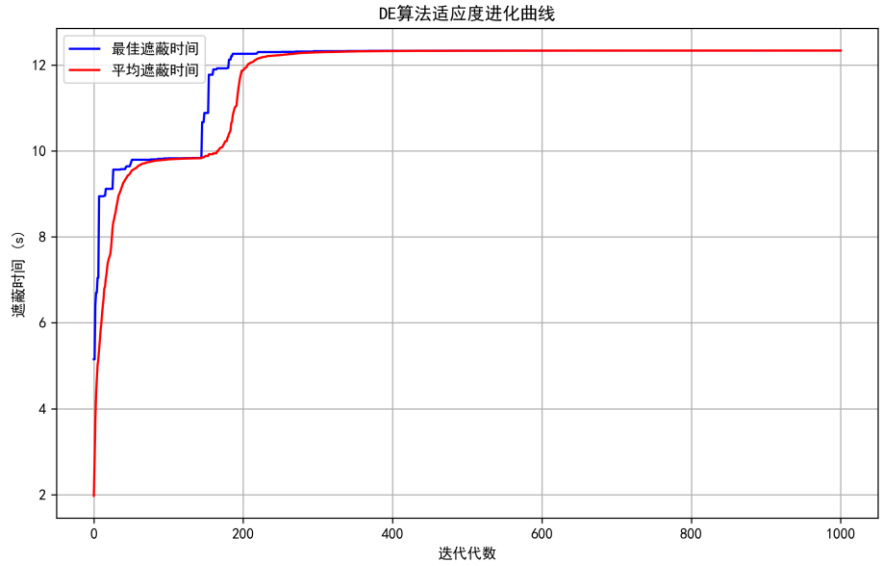


图 8DE 算法遮蔽时间适应度进化曲线

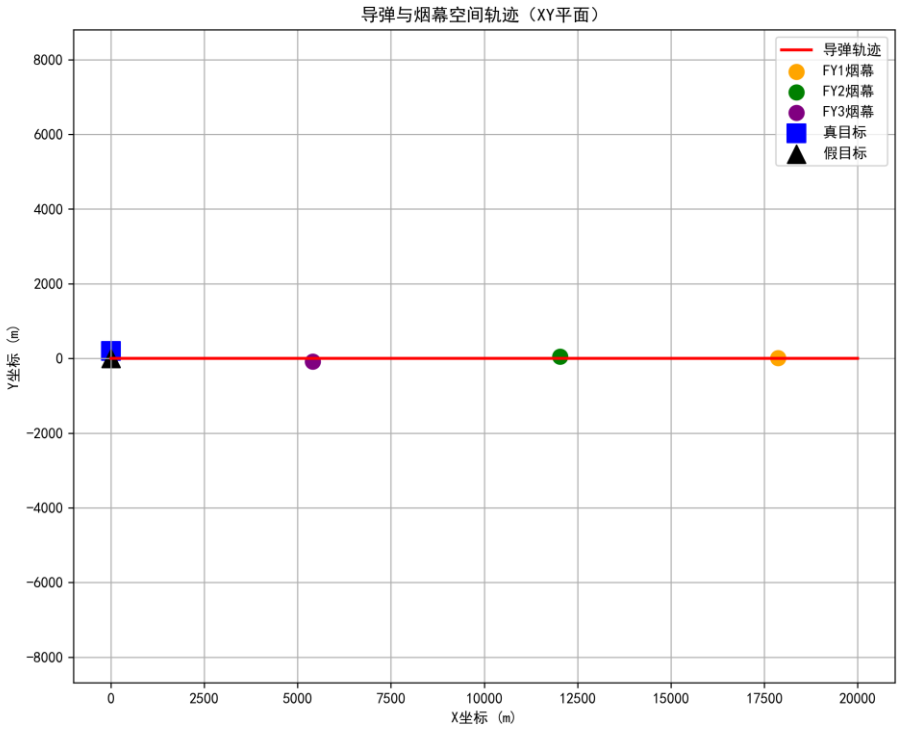


图 9 导弹与烟幕空间轨迹（XY 平面）分布示意图

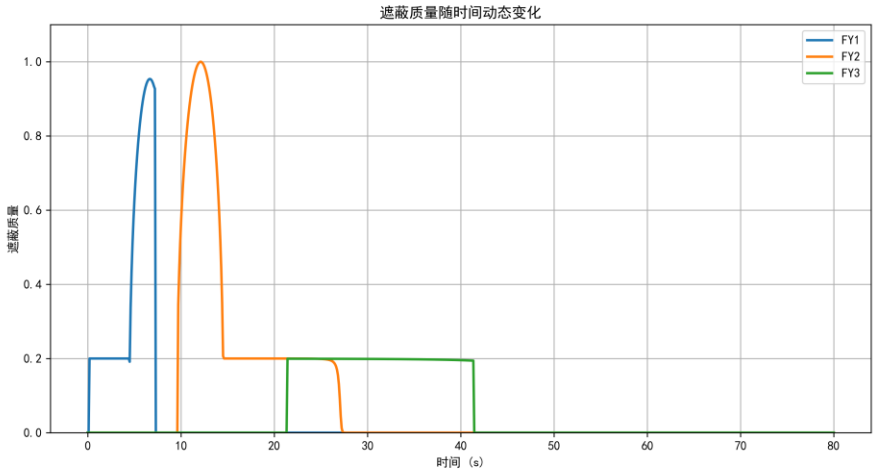


图 10FY1、FY2、FY3 遮蔽质量随时间动态变化曲线

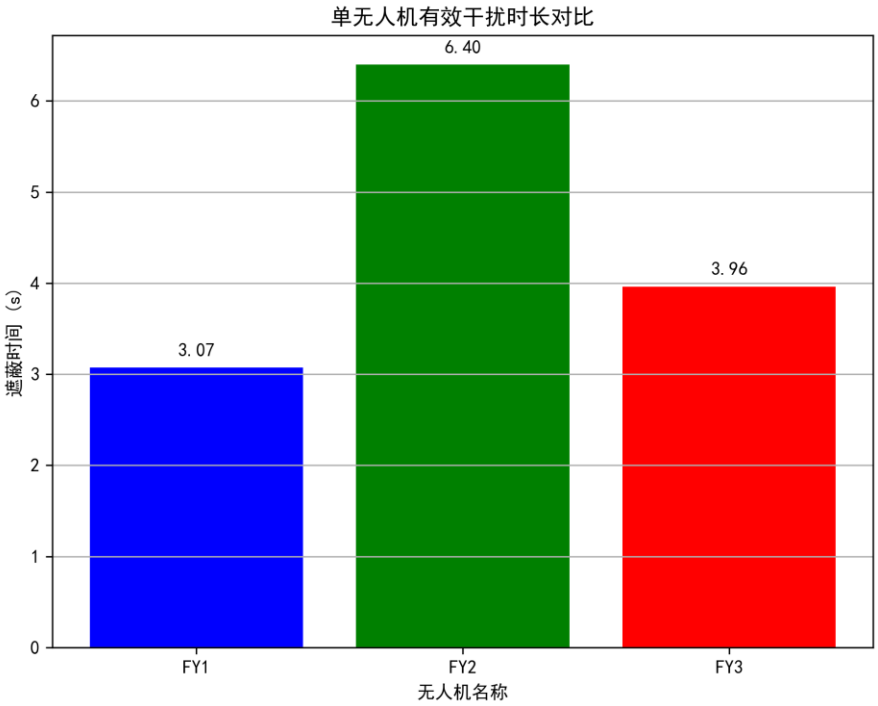


图 11 单无人机有效干扰时长对比图

5.5 问题五模型的建立与求解

5.5.1 模型的建立

步骤一：参照前面，建立五个无人机和三个导弹的运动方程，此处省略

步骤二：采取分层优化算法

第一阶段首先优化时间参数，固定无人机v_u, θ，仅优化烟幕弹的投放和爆炸时间，利用优化器求解当前策略的遮蔽质量函数$Q(t)$在时间 t 上的积分；第二阶段，以第一阶段结果为初始值，优化所有的参数(v_u, θ, t_1, t_2)，采用差分进化的算法，最终输出最优投放策略:

表 3 问题 5 最终结果

无人机	烟幕弹编号	飞行速度(m/s)	航向角(°)	爆开位置 X(m)	爆开位置 Y(m)	爆开位置 Z(m)
FY1	1.0	104.4	319.6	19420.7	-1379.8	755.8
FY1	2.0	105.6	169.4	15161.0	493.7	173.5
FY2	1.0	123.4	241.8	11444.7	363.5	1363.0
FY2	2.0	92.4	262.1	11986.9	1305.7	1394.8
FY3	1.0	130.3	123.9	5795.2	-2695.3	686.9
FY3	2.0	132.9	116.8	4717.5	-456.3	59.0

FY4	1.0	101.2	153.1	9408.0	2807.9	972.4
FY4	2.0	73.5	52.4	11704.5	2913.4	965.6
FY5	1.0	126.9	53.7	13956.4	-700.0	1071.8
FY5	2.0	131.7	264.6	12814.0	-3967.6	1120.1

5.5.2 问题五结果

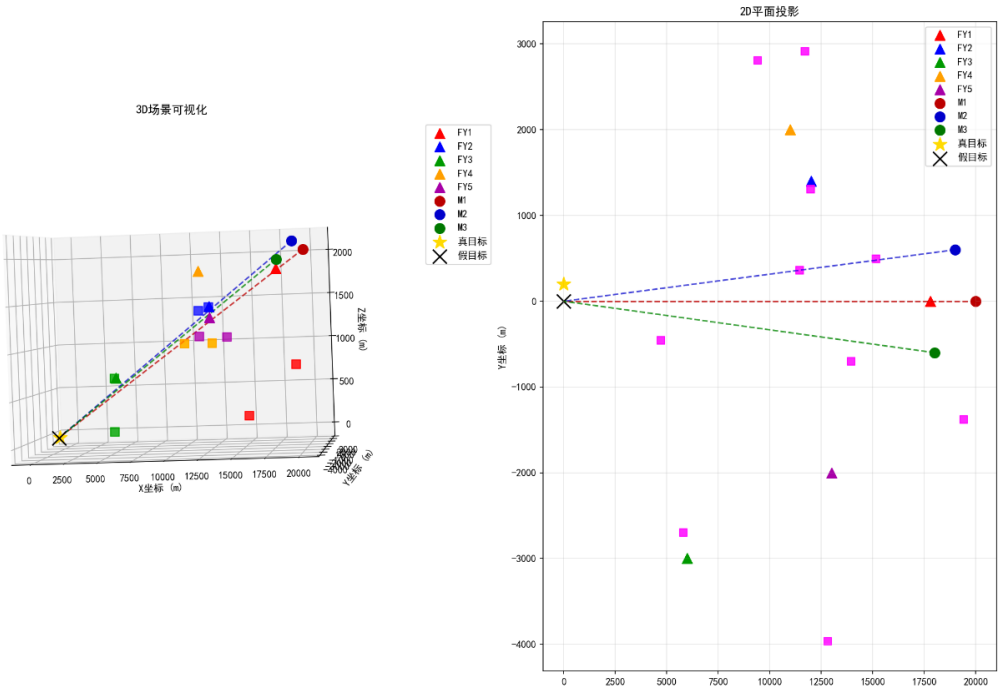


图 12 问题 5 结果位置示意图

五、模型总结

6.1 模型优点

遮蔽质量函数设计科学，本文模型将真目标是否被遮蔽的离散条件转化为0-1 连续的遮蔽质量函数，通过圆锥半顶角与向量夹角的关系量化遮蔽效果，既避免了离散条件下的突变问题，又能精准反映不同时刻的遮蔽程度，为后续梯度优化、积分计算提供平滑基础。

另外，算法适配性强且结果可靠，针对不同问题的参数维度选择适配算法（如问题二用“网格搜索 + 梯度下降”、问题三至五用“差分进化 + 局部精细化”），在保证全局寻优能力的同时提升解的精度；且通过多次迭代验证与灵敏度分析，确保模型输出结果稳定可信。

此外，本文模型的多场景兼容性好，从单无人机单弹到多无人机多弹、从单导弹干扰到多导弹协同防护，模型框架可灵活扩展，运动方程、约束条件的设定能适配不同任务需求，具备较强的通用性。

6.2 模型缺点

本模型的第一个问题是对于真目标简化存在偏差，除问题一考虑真目标圆柱

体形态外，其余问题均将其简化为质点（坐标 (0,200,5)），未完全贴合真目标实际空间形态，可能导致边缘区域遮蔽效果计算不准确，存在一定理论误差。

另外，差分进化算法主观性较强，算法的种群规模、变异因子、交叉概率等参数需人工设定（如问题四设种群规模 50、变异因子 0.8），参数选择依赖经验，可能影响高维参数空间的寻优效率；且局部精细化阶段的收敛性易受初始值影响，存在陷入局部最优的风险。

此外，模型未能考虑复杂外部干扰，模型假设无极端天气、电磁干扰等外部因素，且物体运动均按理想规律进行，但实际场景中风力、气流可能改变烟幕下沉轨迹，电磁干扰可能影响无人机与烟幕弹协同，导致模型在复杂环境下的适用性受限。

6.3 模型推广

本模型在军事与民用领域均有广泛推广价值，可结合场景需求灵活适配。军事层面，当前聚焦无人机烟幕弹干扰单一导弹，未来可拓展至装甲车辆、舰艇等多平台，依据各平台运动特性（如舰艇水面机动、装甲车越野行驶）调整投放与起爆计算逻辑；还能融入红外、激光干扰弹，构建“烟幕 + 光电”复合干扰模型，针对不同制导导弹设计策略，搭配卡尔曼滤波等算法预判导弹轨迹，提前规划投放窗口，提升防护有效性。

民用场景中，应急救援时可借模型规划无人机烟幕投放，在火灾、化学泄漏现场构建屏障，为人员转移开辟安全通道；大型活动安保或重要设施防护中，能精准计算烟幕覆盖范围，实现关键区域临时隐私遮蔽，防范非法航拍。

模型还可持续优化，后续可引入强化学习等智能算法，减少参数设定主观性，提升高维空间寻优效率；结合真实环境数据，增加风力、电磁干扰等随机因素概率模型，搭配数字孪生技术搭建仿真平台，实现投放策略实时推演，进一步扩大应用边界。

六、附录

附录 1

介绍：使用的软件工具

论文写作软件 :Microsoft Word
编程工具:PyCharm、MATLAB
使用 Python 库:NumPy、Pandas、Matplotlib 等
声明：本论文

附录 2

介绍：支撑材料目录，问题 1 到 4 的代码

- **T2.py**: 优化单个无人机针对虚假目标进行烟幕部署的参数，计算导弹和烟幕轨迹、随时间的覆盖质量，并生成多个可视化图。它采用差分进化法和局部优化。
- **T3.py**: 优化单架无人机进行多次烟幕部署（隐含三次烟幕）的部署参数（速度、角度、时机）。它使用带约束的 differential_evolution 和 minimize，并执行分阶段优化以找到最佳策略，将结果保存到 Excel 并可视化场景。
- **T4.py**: 优化三架不同无人机协同干扰导弹的烟幕部署策略。它使用差分进化法，计算总覆盖积分和个体覆盖积分，生成多个复杂的可视化图（2D、3D 轨迹、高度、随时间的质量），并将结果保存到 Excel。这是多无人机策略的增强版本。
- **T1.m**: 一个 MATLAB 脚本，为简化的单导弹、单烟幕场景执行解析推导和可视化。它解析推导出覆盖的锥形方程，计算特定时间的位置，并可视化该锥形与水平面（Z=10）的交集，包括导弹和烟幕的投影位置。

附录 3 问题一

介绍：该代码是 matlab 编写的。

```
%%
g = 9.8;
FY1_v = 120;
FY1_t = 1.5;
Smoke_t = 3.6;
M1_v = 300;
Smoke_v = 3;
P = [0, 200, 0];

%% 首先计算烟幕爆开的位置 S
x1 = FY1_t * FY1_v;
x2 = FY1_v * Smoke_t;
h = 1/2 * g * Smoke_t^2;
```

```
Smoke_x = 17800 - x1 - x2;
Smoke_y = 0;
Smoke_z = 1800 - h;
Smoke_r = 10;

S = [Smoke_x, Smoke_y, Smoke_z];
disp(S);
%% 计算烟雾弹爆开时导弹的位置 M1
M1_S = M1_v * (FY1_t + Smoke_t);
c = 20000 / sqrt(20000^2 + 2000^2);
s = 2000 / sqrt(20000^2 + 2000^2);
M1_x = 20000 - M1_S * c;
M1_y = 0;
M1_z = 2000 - M1_S * s;
M1 = [M1_x, M1_y, M1_z];

M1_v_x = M1_v * c;
M1_v_z = M1_v * s;
disp(M1);
%% 求圆锥
syms t x y z real
M_x = M1_x - M1_v_x * t;
M_y = 0;
M_z = M1_z - M1_v_z * t;
M = [M_x, M_y, M_z];

S_x = Smoke_x;
S_y = 0;
S_z = Smoke_z - Smoke_v * t;
S = [S_x, S_y, S_z];


d = S - M;
L = norm(d);
MT = sqrt(L^2 - Smoke_r^2);
cos_theta = MT/L;

Q = [x,y,z];
MQ = Q-M;

cone_equation = ( dot(MQ, d)/ (norm(MQ)*norm(d)) ) - cos_theta == 0;

cone_equation = simplify(cone_equation);
```

```
disp('圆锥的解析方程为：');
disp(cone_equation);

cone_poly = expand(lhs(cone_equation));

disp('圆锥的多项式形式：');
disp(cone_poly == 0);

%%  第一次包裹时间是 2.9140
t_out = (M1_x - Smoke_x) / M1_v_x;

%% 画图

T = 2.9290;  % 给定时间
Z = 10; % 给定高度平面

% 假设以下参数已定义（根据实际数据修改）
% M1_x, M1_v_x, M1_z, M1_v_z, Smoke_x, Smoke_z, Smoke_v, Smoke_r

M_x_t = M1_x - M1_v_x * T;
M_z_t = M1_z - M1_v_z * T;
S_x_t = Smoke_x;
S_z_t = Smoke_z - Smoke_v * T;

M_T = [M_x_t, 0, M_z_t];
S_T = [S_x_t, 0, S_z_t];

d_vector = S_T - M_T;
d_norm = norm(d_vector);

% 圆锥半顶角的正弦和余弦
sin_theta = Smoke_r / d_norm;
cos_theta = sqrt(1 - sin_theta^2); % MT/L



syms x y real

Q = [x, y, Z];
M_T_vec = [M_x_t, 0, M_z_t];
d_vec = d_vector;

% 圆锥方程：(MQ・d)/(|MQ||d|) = cos_theta
MQ = Q - M_T_vec;
norm_MQ_sq = (x - M_x_t)^2 + y^2 + (Z - M_z_t)^2;
```

```
dot_MQ_d = dot(MQ, d_vec);
norm_d_sq = norm(d_vec)^2;

cone_eq = dot_MQ_d^2 - cos_theta^2 * norm_MQ_sq * norm_d_sq;
cone_eq_expanded = expand(cone_eq);

disp('在 Z=10 平面，T 时刻的椭圆方程为：');
disp(cone_eq_expanded == 0);


% 系数提取
syms x y real;

% 使用求导方法提取系数
A_coeff = diff(cone_eq_expanded, x, 2) / 2;
A_coeff = simplify(A_coeff);
B_coeff = diff(cone_eq_expanded, y, 2) / 2;
B_coeff = simplify(B_coeff);
C_temp = diff(cone_eq_expanded, x);
C_coeff = simplify(C_temp - 2*A_coeff*x);
D_coeff = subs(cone_eq_expanded, {x, y}, {0, 0});
D_coeff = simplify(D_coeff);

disp('提取的系数:');
fprintf('A = %s\n', char(A_coeff));
fprintf('B = %s\n', char(B_coeff));
fprintf('C = %s\n', char(C_coeff));
fprintf('D = %s\n', char(D_coeff));

[A_num, A_den] = numden(A_coeff);
[B_num, B_den] = numden(B_coeff);
[C_num, C_den] = numden(C_coeff);
[D_num, D_den] = numden(D_coeff);


% 使用提取的系数进行椭圆分析
A = A_coeff;
B = B_coeff;
C = C_coeff;
D = D_coeff;

% 配方转化为标准椭圆方程
eq = A*x^2 + B*y^2 + C*x + D == 0;
```

```
% 对 x 配方：A(x-h)^2 + B y^2 = E
h = -C/(2*A);   % x 方向中心
E = (C^2)/(4*A) - D;   % 配方后的常数项

% 标准方程：(x-h)^2/(E/A) + y^2/(E/B) = 1
a_sq = E/A;   % x 方向的半轴平方
b_sq = E/B;   % y 方向的半轴平方

% 检查是否为椭圆（a_sq > 0, b_sq > 0）
if double(a_sq) > 0 && double(b_sq) > 0
    a = sqrt(a_sq);
    b = sqrt(b_sq);

    fprintf('\n 椭圆参数:\n');
    fprintf('椭圆中心  (h, k): (%s, 0)\n', char(vpa(h, 10)));
    fprintf('半长轴  a: %s\n', char(vpa(a, 10)));
    fprintf('半短轴  b: %s\n', char(vpa(b, 10)));

    % 转换为数值用于绘图
    h_num = double(h);
    a_num = double(a);
    b_num = double(b);

    % 生成椭圆点
    theta = linspace(0, 2*pi, 1000);
    x_vals = h_num + a_num * cos(theta);
    y_vals = b_num * sin(theta);

    % 生成额外的圆：圆心(0,200)，半径 7
    circle_theta = linspace(0, 2*pi, 1000);
    circle_x = 0 + 7 * cos(circle_theta);
    circle_y = 200 + 7 * sin(circle_theta);

    % 计算导弹和烟幕在 Z=10 平面的投影位置
    M_x_t = M1_x - M1_v_x * T;
    M_z_t = M1_z - M1_v_z * T;
    S_x_t = Smoke_x;
    S_z_t = Smoke_z - Smoke_v * T;

    % 绘图
    figure;
    plot(x_vals, y_vals, 'b-', 'LineWidth', 2);
    hold on;
    % 绘制额外的圆
```

```
    plot(circle_x, circle_y, 'r-', 'LineWidth', 2);
    plot(h_num, 0, 'ro', 'MarkerSize', 8, 'MarkerFaceColor', 'r');
    plot(M_x_t, 0, 'g^', 'MarkerSize', 10, 'MarkerFaceColor', 'g');
    plot(S_x_t, 0, 'ms', 'MarkerSize', 10, 'MarkerFaceColor', 'm');
    % 标记新添加圆的圆心
    plot(0, 200, 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k');

    % 添加标注
    text(h_num, 0, '  椭圆中心', 'VerticalAlignment', 'bottom');
    text(M_x_t, 0, '    导弹位置', 'VerticalAlignment', 'bottom');
    text(S_x_t, 0, '    烟幕中心', 'VerticalAlignment', 'bottom');
    text(0, 200, '    新增圆圆心', 'VerticalAlignment', 'bottom');

    xlabel('x 坐标');
    ylabel('y 坐标');
    title('2.929 时刻，Z=10 平面上的烟幕椭圆投影及真目标');
    legend('烟幕椭圆', '真目标(0,200),r=7', '椭圆中心', '导弹位置', '烟幕中心', '
真目标', 'Location', 'best');
    grid on;
    axis equal;   % 保持横纵比一致

    % 调整坐标轴范围：Y 轴扩大显示范围（关键修改）
    % 计算当前 Y 轴元素的最大最小值
    y_min = min([y_vals, circle_y, 0, 200]);
    y_max = max([y_vals, circle_y, 0, 200]);
    % 扩展 Y 轴范围（上下各增加当前范围的 50%，确保足够宽松）
    y_range = y_max - y_min;
    y_min_expanded = y_min - 0.5 * y_range;   % 向下扩展 50%
    y_max_expanded = y_max + 0.5 * y_range;   % 向上扩展 50%

    % X 轴范围保持自适应，Y 轴用扩展后的范围
    x_min = min([x_vals, circle_x, M_x_t, S_x_t, 0]) - 10;
    x_max = max([x_vals, circle_x, M_x_t, S_x_t, 0]) + 10;
    axis([x_min, x_max, y_min_expanded, y_max_expanded]);

else
    error('方程不构成椭圆（a² 或 b² 非正）');
end
```

附录 3 问题二

介绍：该代码是 python 语言编写的

```
import numpy as np
from scipy.optimize import minimize, Bounds
import matplotlib.pyplot as plt
import pandas as pd
from datetime import datetime
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False  # 解决负号显示问题

g = 9.8
smoke_radius = 10  # 烟幕半径
smoke_sink_speed = 3  # 烟幕下沉速度
missile_speed = 300  # 导弹速度

FY1_initial = np.array([17800, 0, 1800])
M1_initial = np.array([20000, 0, 2000])
target_point = np.array([0, 200, 0])  # 真
fake_target = np.array([0, 0, 0])  # 假


def calculate_missile_direction():
    direction = fake_target - M1_initial
    return direction / np.linalg.norm(direction)
missile_direction = calculate_missile_direction()

def missile_position(t):
    return M1_initial + missile_speed * t * missile_direction


def calculate_smoke_burst(params):
    v_u, theta_deg, t1, t2 = params
    theta = np.deg2rad(theta_deg)

    uav_velocity = v_u * np.array([np.cos(theta), np.sin(theta), 0])

    drop_position = FY1_initial + uav_velocity * t1

    horizontal_displacement = uav_velocity * t2
    vertical_displacement = 0.5 * g * t2 ** 2
    burst_position = drop_position + horizontal_displacement - np.array([0, 0,
vertical_displacement])

    burst_time = t1 + t2
```

```
    return burst_position, burst_time


def calculate_cover_quality_improved(M, S, P):
    try:
        MS = S - M
        MP = P - M

        distance_MS = np.linalg.norm(MS)

        # 导弹在烟幕内
        if distance_MS < smoke_radius:
            return 1.0 - (distance_MS / smoke_radius) ** 2  # 中心区域质量更
高

        cos_beta = np.dot(MP, MS) / (np.linalg.norm(MP) * distance_MS)

        critical_angle = np.arcsin(smoke_radius / distance_MS)
        cos_critical = np.cos(critical_angle)

        if cos_beta >= cos_critical:
            relative_position = (cos_beta - cos_critical) / (1 - cos_critical + 1e-
10)
            quality = np.sqrt(relative_position)
        else:
            falloff = 1.0 - (cos_critical - cos_beta) / (cos_critical + 1.0)
            quality = max(0, 0.2 * falloff ** 3)  # 快速衰减

        return min(1.0, max(0.0, quality))

    except:
        return 0.0


def objective_function_improved(params):
    try:
        v_u, theta_deg, t1, t2 = params

        # 更严格的参数边界检查
        if not (70 <= v_u <= 140 and 0 <= theta_deg <= 360 and
                0 <= t1 <= 10 and 0 <= t2 <= 20 and t1 + t2 <= 30):
            return 10000 + np.sum(np.abs(np.clip(params, [70, 0, 0, 0], [140, 360,
10, 20]) - params))
```

```
        theta = np.deg2rad(theta_deg)

        uav_velocity = v_u * np.array([np.cos(theta), np.sin(theta), 0])

        drop_position = FY1_initial + uav_velocity * t1
        horizontal_displacement = uav_velocity * t2
        vertical_displacement = 0.5 * g * t2 ** 2
        burst_position = drop_position + horizontal_displacement - np.array([0, 0,
vertical_displacement])
        burst_time = t1 + t2

        if burst_position[2] < 0:
            return 10000 + abs(burst_position[2]) * 100   # 根据地下深度增加
惩罚

        total_integral = 0
        max_dt = 0.01
        min_dt = 0.001
        t_current = max(burst_time, 0)
        t_end = min(burst_time + 20, 40)

        if t_current >= t_end:
            return 10000

        prev_quality = 0
        iteration_count = 0
        max_iterations = 10000   # 防止无限循环

        while t_current < t_end and iteration_count < max_iterations:
            dt = max(min_dt, min(max_dt, 0.1 / (abs(prev_quality - 0.5) + 0.1)))

            # 导弹位置
            M_t = missile_position(t_current)

            sink_distance = smoke_sink_speed * (t_current - burst_time)
            S_t = burst_position - np.array([0, 0, sink_distance])

            if S_t[2] < 0:
                t_current += dt
                iteration_count += 1
                continue

            # 计算平滑遮蔽质量
```

```
                quality = calculate_cover_quality_improved(M_t, S_t, target_point)
                total_integral += quality * dt

                prev_quality = quality
                t_current += dt
                iteration_count += 1

            print(f"参数：v_u={v_u:.1f}，theta={theta_deg:.1f}°，t1={t1:.3f}，
t2={t2:.3f} -> 积分: {total_integral:.6f}")
            return -total_integral

        except Exception as e:
            print(f"目标函数计算错误: {e}")
            return 10000


def optimize_smoke_strategy_advanced():
    bounds = [(70, 140), (0, 360), (0, 10), (0, 20)]

    # 使用更密集的初始点网格（集中在可能的最优区域）
    initial_guesses = []

    # 第一层：粗网格搜索
    for v in np.linspace(80, 130, 3):
        for theta in np.linspace(170, 190, 5):
            for t1 in np.linspace(1, 3, 3):
                for t2 in np.linspace(4, 7, 4):
                    initial_guesses.append([v, theta, t1, t2])

    # 第二层：更精细的网格
    for v in np.linspace(100, 120, 2):
        for theta in np.linspace(175, 185, 4):
            for t1 in np.linspace(1.5, 2.5, 3):
                for t2 in np.linspace(5, 6, 3):
                    initial_guesses.append([v, theta, t1, t2])

    np.random.seed(42)
    for _ in range(5):
        initial_guesses.append([
            np.random.uniform(70, 140),
            np.random.uniform(0, 360),
            np.random.uniform(0, 10),
            np.random.uniform(0, 20)
        ])
```

```
best_result = None
best_value = float('inf')
results = []
print(f"总共尝试 {len(initial_guesses)} 个初始点")
for i, x0 in enumerate(initial_guesses):
    try:
        # 使用不同的优化方法
        result = minimize(
            objective_function_improved,
            x0,
            method='L-BFGS-B',  # 使用 L-BFGS-B 方法
            bounds=bounds,
            options={
                'maxiter': 200,
                'disp': False,
                'ftol': 1e-8,
                'gtol': 1e-8
            }
        )

        results.append((result.x, result.fun, result.success))

        if result.fun < best_value and result.success:
            best_result = result
            best_value = result.fun
        else:
            return 0

    except Exception as e:
        print(f"✗ 优化失败: {e}")
        continue

# 对结果进行局部精细化
if best_result is not None:
    try:
        refined_result = minimize(
            objective_function_improved,
            best_result.x,
            method='Nelder-Mead',  # 单纯形法对局部精细化效果好
            options={
                'maxiter': 100,
                'xatol': 1e-6,
```

```
                        'fatol': 1e-6,
                        'disp': True
                    }
                )

                if refined_result.fun < best_value:
                    best_result = refined_result
                    best_value = refined_result.fun
            except Exception as e:
                print(f"✗ 精细化失败: {e}")

    if best_result is not None:
        best_params = best_result.x
        best_value = -best_result.fun
        return best_params, best_value
    else:
        return None, None


def validate_and_visualize(params):
    v_u, theta_deg, t1, t2 = params
    burst_position, burst_time = calculate_smoke_burst(params)

    # 计算遮蔽时间积分
    total_integral = -objective_function_improved(params)
    print(f"遮蔽时间积分: {total_integral:.6f}")

    # 生成时间序列数据用于绘图
    times = np.linspace(burst_time, burst_time + 30, 1000)
    qualities = []
    missile_positions = []
    smoke_positions = []

    for t in times:
        M_t = missile_position(t)
        sink_distance = smoke_sink_speed * (t - burst_time)
        S_t = burst_position - np.array([0, 0, sink_distance])
        quality = calculate_cover_quality_improved(M_t, S_t, target_point)

        qualities.append(quality)
        missile_positions.append(M_t)
        smoke_positions.append(S_t)
    plt.figure(figsize=(12, 8))
```

```
    plt.subplot(2, 2, 1)
    plt.plot(times - burst_time, qualities, 'b-', linewidth=2)
    plt.xlabel('爆开后时间 (s)')
    plt.ylabel('遮蔽质量')
    plt.title('遮蔽质量随时间变化')
    plt.grid(True, alpha=0.3)

    plt.subplot(2, 2, 2)
    missile_x = [pos[0] for pos in missile_positions]
    missile_y = [pos[1] for pos in missile_positions]
    smoke_x = [pos[0] for pos in smoke_positions]
    smoke_y = [pos[1] for pos in smoke_positions]

    plt.plot(missile_x, missile_y, 'r-', label='导弹轨迹')
    plt.plot(smoke_x, smoke_y, 'g--', label='烟幕轨迹')
    plt.scatter(target_point[0], target_point[1], c='blue', s=100, label='目标')
    plt.scatter(burst_position[0], burst_position[1], c='orange', s=100, label='爆开
位置')
    plt.xlabel('X 坐标 (m)')
    plt.ylabel('Y 坐标 (m)')
    plt.title('导弹和烟幕轨迹')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.subplot(2, 2, 3)
    altitudes = [pos[2] for pos in missile_positions]
    smoke_altitudes = [pos[2] for pos in smoke_positions]
    plt.plot(times - burst_time, altitudes, 'r-', label='导弹高度')
    plt.plot(times - burst_time, smoke_altitudes, 'g--', label='烟幕高度')
    plt.xlabel('爆开后时间 (s)')
    plt.ylabel('高度 (m)')
    plt.title('高度变化')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.subplot(2, 2, 4)
    effective_qualities = [q for q in qualities if q > 0.1]
    plt.hist(effective_qualities, bins=20, alpha=0.7, color='purple')
    plt.xlabel('遮蔽质量')
    plt.ylabel('频率')
    plt.title('遮蔽质量分布')
    plt.grid(True, alpha=0.3)
```

```
    plt.tight_layout()
    plt.savefig('smoke_cover_analysis.png', dpi=300, bbox_inches='tight')
    plt.show()

    return total_integral


def test_optimal_region():
    test_points = []
    for v in [100, 110, 120, 130]:
        for theta in [175, 180, 185]:
            for t1 in [1.5, 2.0, 2.5]:
                for t2 in [5.0, 5.5, 6.0]:
                    test_points.append([v, theta, t1, t2])

    results = []
    for params in test_points:
        value = -objective_function_improved(params)
        results.append((params, value))
        print(f"测试点 {params} -> 积分: {value:.6f}")

    best_test = max(results, key=lambda x: x[1])
    print(f"\n 最佳测试点: {best_test[0]} -> 积分: {best_test[1]:.6f}")

    return best_test[0], best_test[1]


if __name__ == "__main__":
    test_params, test_value = test_optimal_region()
    best_params, best_value = optimize_smoke_strategy_advanced()

    if best_params is not None:
        final_value = validate_and_visualize(best_params)

        v_u, theta_deg, t1, t2 = best_params
        burst_position, burst_time = calculate_smoke_burst(best_params)
        print(f"• 飞行速度: {v_u:.6f} m/s")
        print(f"• 航向角: {theta_deg:.6f}° ")
        print(f"• 投放时间: {t1:.6f} s 后")
        print(f"• 起爆时间: {t2:.6f} s 后")
        print(f"• 烟幕爆开时间: {burst_time:.6f} s")
        print(f" • 烟幕爆开位置: ({burst_position[0]:.6f},
{burst_position[1]:.6f}, {burst_position[2]:.6f})")
        print(f"• 预计最大遮蔽积分: {final_value:.6f}")
```

```
    print("完成")
```

附录 3 问题三

介绍：该代码是 python 语言编写的

```
import numpy as np
from scipy.optimize import differential_evolution, minimize
import matplotlib.pyplot as plt
import pandas as pd
from datetime import datetime

plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False

# 物理常数
g = 9.8  # 重力加速度
smoke_radius = 10  # 烟幕半径
smoke_sink_speed = 3  # 烟幕下沉速度
missile_speed = 300  # 导弹速度

# 初始位置
FY1_initial = np.array([17800, 0, 1800])
M1_initial = np.array([20000, 0, 2000])
target_point = np.array([0, 200, 0])  # 真目标
fake_target = np.array([0, 0, 0])  # 假目标


def calculate_missile_direction():
    """计算导弹飞行方向（指向假目标）"""
    direction = fake_target - M1_initial
    return direction / np.linalg.norm(direction)


missile_direction = calculate_missile_direction()


def missile_position(t):
    return M1_initial + missile_speed * t * missile_direction


def calculate_smoke_burst_single(v_u, theta_deg, t1, t2):
    theta = np.deg2rad(theta_deg)
    uav_velocity = v_u * np.array([np.cos(theta), np.sin(theta), 0])
    drop_position = FY1_initial + uav_velocity * t1
```

```
    horizontal_displacement = uav_velocity * t2
    vertical_displacement = 0.5 * g * t2 ** 2
    burst_position = drop_position + horizontal_displacement - np.array([0, 0, vertical_displacement])
    burst_time = t1 + t2
    return burst_position, burst_time


def calculate_cover_quality_improved(M, S, P):
    try:
        MS = S - M
        MP = P - M
        distance_MS = np.linalg.norm(MS)

        if distance_MS < smoke_radius:
            return 1.0 - (distance_MS / smoke_radius) ** 2

        if distance_MS < 1e-12 or np.linalg.norm(MP) < 1e-12:
            return 0.0

        cos_beta = np.dot(MP, MS) / (np.linalg.norm(MP) * distance_MS)
        critical_angle = np.arcsin(smoke_radius / distance_MS)
        cos_critical = np.cos(critical_angle)

        if cos_beta >= cos_critical:
            relative_position = (cos_beta - cos_critical) / (1 - cos_critical + 1e-10)
            quality = np.sqrt(relative_position)
        else:
            falloff = 1.0 - (cos_critical - cos_beta) / (cos_critical + 1.0)
            quality = max(0, 0.2 * falloff ** 3)

        return min(1.0, max(0.0, quality))
    except:
        return 0.0


def calculate_single_smoke_cover_time(v_u, theta_deg, t1, t2, quality_threshold=0.1):
    burst_pos, burst_time = calculate_smoke_burst_single(v_u, theta_deg, t1, t2)

    total_time = 0
    dt = 0.01
    t_current = burst_time
```

```
    max_simulation_time = 30  # 最大仿真时间

    while t_current < burst_time + max_simulation_time:
        M_t = missile_position(t_current)
        sink_distance = smoke_sink_speed * (t_current - burst_time)
        S_t = burst_pos - np.array([0, 0, sink_distance])

        if S_t[2] < 0:  # 烟幕已经沉到地面以下
            break

        quality = calculate_cover_quality_improved(M_t, S_t, target_point)

        if quality > quality_threshold:
            total_time += dt

        t_current += dt

    return total_time


def check_smoke_constraints(t1_params):
    t1_sorted = np.sort(t1_params)
    penalty = 0

    for i in range(len(t1_sorted) - 1):
        time_gap = t1_sorted[i + 1] - t1_sorted[i]
        if time_gap < 1.0:
            penalty += 1000 * (1.0 - time_gap) ** 2

    return penalty


def check_physical_constraints(params):
    v_u, theta_deg, t1_1, t2_1, t1_2, t2_2, t1_3, t2_3 = params
    penalty = 0

    for t2 in [t2_1, t2_2, t2_3]:
        if t2 > 8:  # 如果起爆时间超过 8 秒，惩罚
            penalty += 100 * (t2 - 8) ** 2

    # 检查投放时间顺序
    t1_params = np.array([t1_1, t1_2, t1_3])
    t1_sorted = np.sort(t1_params)
    for i in range(len(t1_sorted) - 1):
```

```
            if t1_sorted[i + 1] - t1_sorted[i] < 1.0:
                penalty += 500

    return penalty


def objective_function_all_params(params):
    v_u, theta_deg, t1_1, t2_1, t1_2, t2_2, t1_3, t2_3 = params

    # 检查参数边界
    if not (70 <= v_u <= 140):
        return 10000 + abs(v_u - 70 if v_u < 70 else v_u - 140) * 100

    # 检查物理约束
    physical_penalty = check_physical_constraints(params)

    # 如果严重违反约束，直接返回大惩罚值
    if physical_penalty > 100:
        return physical_penalty

    bursts = []
    for t1, t2 in [(t1_1, t2_1), (t1_2, t2_2), (t1_3, t2_3)]:
        if t1 < 0 or t1 > 30 or t2 < 0 or t2 > 20:
            return 10000  # 参数越界

        burst_pos, burst_time = calculate_smoke_burst_single(v_u, theta_deg, t1,
t2)
        bursts.append((burst_pos, burst_time))

    total_integral = 0
    max_dt = 0.02
    min_dt = 0.005
    t_start = min(burst_time for _, burst_time in bursts)
    t_end = max(burst_time for _, burst_time in bursts) + 30
    t_current = t_start
    prev_quality = 0
    iteration_count = 0
    max_iterations = 3000

    while t_current < t_end and iteration_count < max_iterations:
        dt = max(min_dt, min(max_dt, 0.1 / (abs(prev_quality - 0.5) + 0.1)))
        M_t = missile_position(t_current)
        max_quality = 0
```

```
            for burst_pos, burst_time in bursts:
                sink_distance = smoke_sink_speed * (t_current - burst_time)
                S_t = burst_pos - np.array([0, 0, sink_distance])
                if S_t[2] < 0:
                    continue
                quality = calculate_cover_quality_improved(M_t, S_t, target_point)
                max_quality = max(max_quality, quality)

            total_integral += max_quality * dt
            prev_quality = max_quality
            t_current += dt
            iteration_count += 1

        return -total_integral + physical_penalty


def efficient_optimization():
        bounds = [
            (100, 140),    # v_u
            (170, 190),    # theta_deg
            (0, 10), (1, 6),
            (0, 10), (1, 6),
            (0, 10), (1, 6)
        ]

        initial_guess = [120.0, 180.0, 1.5, 3.0, 2.5, 3.5, 3.5, 4.0]

        result_de = differential_evolution(
            objective_function_all_params,
            bounds,
            strategy='best1bin',
            maxiter=50,
            popsize=15,
            tol=1e-4,
            mutation=(0.5, 1),
            recombination=0.7,
            seed=42,
            disp=True,
            polish=False
        )

        print(f"差分进化结果: {-result_de.fun:.6f}")

        result_local = minimize(
```

```
            objective_function_all_params,
            result_de.x,
            method='L-BFGS-B',
            bounds=bounds,
            options={
                'maxiter': 100,
                'ftol': 1e-6,
                'gtol': 1e-6,
                'disp': True
            }
        )

        best_params = result_local.x
        best_value = -result_local.fun

        # 解析最优参数
        v_u, theta_deg = best_params[0], best_params[1]
        smoke_params = best_params[2:].reshape(3, 2)
        t1_params = smoke_params[:, 0]

        # 检查约束
        time_gaps = np.diff(np.sort(t1_params))
        min_gap = np.min(time_gaps) if len(time_gaps) > 0 else 0

        print("\n" + "=" * 50)
        print("===  最优策略  ===")
        print(f"无人机速度: {v_u:.6f} m/s")
        print(f"无人机航向: {theta_deg:.6f}°  ")
        print(f"总遮蔽积分: {best_value:.6f}")
        print(f"最小投放间隔: {min_gap:.6f}s (要求≥1.0s)")

        individual_cover_times = []
        for i, (t1, t2) in enumerate(smoke_params):
            cover_time = calculate_single_smoke_cover_time(v_u, theta_deg, t1, t2)
            individual_cover_times.append(cover_time)
            print(f"烟幕弹  {i + 1}  单独遮蔽时间: {cover_time:.3f}s")

        # 输出烟幕弹详细参数
        results = []
        for i, (t1, t2) in enumerate(smoke_params):
            burst_pos, burst_time = calculate_smoke_burst_single(v_u, theta_deg, t1,
t2)

            fall_distance = 0.5 * g * t2 ** 2
```

```
            initial_height = FY1_initial[2]  # 无人机初始高度

            print(f"  烟幕弹 {i + 1}:")
            print(f"    • 投放时间: {t1:.6f}s")
            print(f"    • 起爆时间: {t2:.6f}s")
            print(f"    • 爆开时间: {burst_time:.6f}s")
            print(f"      • 爆开位置: ({burst_pos[0]:.2f}, {burst_pos[1]:.2f},
{burst_pos[2]:.2f})")
            print(f"    • 下落距离: {fall_distance:.2f}m")
            print(f"    • 初始高度: {initial_height:.2f}m")
            print(f"    • 单独遮蔽时间: {individual_cover_times[i]:.3f}s")

            results.append({
                '烟幕弹序号': i + 1,
                '投放时间(s)': t1,
                '起爆时间(s)': t2,
                '爆开时间(s)': burst_time,
                '爆开 X 坐标(m)': burst_pos[0],
                '爆开 Y 坐标(m)': burst_pos[1],
                '爆开 Z 坐标(m)': burst_pos[2],
                '下落距离(m)': fall_distance,
                '单独遮蔽时间(s)': individual_cover_times[i]
            })

    # 保存结果
    df = pd.DataFrame(results)
    df.to_excel('result3_optimal.xlsx', index=False)
    print("\n 结果已保存到 result3_optimal.xlsx")


    return best_params, best_value, individual_cover_times


def validate_and_visualize_all_smokes(v_u, theta_deg, smoke_params,
individual_cover_times):
    smoke_params = smoke_params.reshape(3, 2)
    bursts = []
    for i, (t1, t2) in enumerate(smoke_params):
        burst_pos, burst_time = calculate_smoke_burst_single(v_u, theta_deg, t1,
t2)
        bursts.append((burst_pos, burst_time))
        print(
            f"烟幕弹 {i + 1}: 投放时间={t1:.3f}s, 起爆时间={t2:.3f}s, 爆开
位置=({burst_pos[0]:.1f}, {burst_pos[1]:.1f}, {burst_pos[2]:.1f})")
```

```
    burst_times = [burst_time for _, burst_time in bursts]
    start_time = min(burst_times)
    end_time = max(burst_times) + 30
    times = np.linspace(start_time, end_time, 2000)

    all_qualities = []
    missile_positions = []
    smoke_positions_all = [[] for _ in range(3)]

    for t in times:
        M_t = missile_position(t)
        missile_positions.append(M_t)

        max_quality = 0
        for i, (burst_pos, burst_time) in enumerate(bursts):
            sink_distance = smoke_sink_speed * (t - burst_time)
            S_t = burst_pos - np.array([0, 0, sink_distance])
            smoke_positions_all[i].append(S_t)

            if S_t[2] >= 0:  # 只考虑地面以上的烟幕
                quality = calculate_cover_quality_improved(M_t, S_t,
target_point)
                max_quality = max(max_quality, quality)

        all_qualities.append(max_quality)

    # 绘制综合结果
    fig = plt.figure(figsize=(16, 12))

    # 颜色定义（修复画图错误）
    colors = ['r', 'g', 'b']  # 使用单字符颜色代码

    # 1. 遮蔽质量随时间变化
    ax1 = plt.subplot(2, 2, 1)
    plt.plot(times - start_time, all_qualities, 'b-', linewidth=2)
    plt.xlabel('时间 (s)')
    plt.ylabel('遮蔽质量')
    plt.title('三枚烟幕弹综合遮蔽质量随时间变化')
    plt.grid(True, alpha=0.3)

    # 标记各烟幕弹爆开时间
    for i, burst_time in enumerate(burst_times):
        plt.axvline(x=burst_time - start_time, color=colors[i], linestyle='--',
```

```
                label=f'烟幕{i + 1}爆开: {burst_time - start_time:.1f}s')
    plt.legend()

    # 2. 导弹和烟幕轨迹（XY 平面）
    ax2 = plt.subplot(2, 2, 2)
    missile_x = [pos[0] for pos in missile_positions]
    missile_y = [pos[1] for pos in missile_positions]
    plt.plot(missile_x, missile_y, 'r-', label='导弹轨迹', linewidth=2)

    for i, smoke_positions in enumerate(smoke_positions_all):
        smoke_x = [pos[0] for pos in smoke_positions]
        smoke_y = [pos[1] for pos in smoke_positions]
        plt.plot(smoke_x, smoke_y, color=colors[i], linestyle='--',
                 label=f'烟幕{i + 1}轨迹', linewidth=2, alpha=0.7)

    plt.scatter(target_point[0], target_point[1], c='orange', s=200, label='目标',
zorder=5)
    for i, (burst_pos, _) in enumerate(bursts):
        plt.scatter(burst_pos[0], burst_pos[1], color=colors[i], s=100,
                    label=f'烟幕{i + 1}爆开位置', zorder=5)

    plt.xlabel('X 坐标 (m)')
    plt.ylabel('Y 坐标 (m)')
    plt.title('导弹和烟幕轨迹（XY 平面）')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # 3. 高度变化
    ax3 = plt.subplot(2, 2, 3)
    altitudes = [pos[2] for pos in missile_positions]
    plt.plot(times - start_time, altitudes, 'r-', label='导弹高度', linewidth=2)

    for i, smoke_positions in enumerate(smoke_positions_all):
        smoke_altitudes = [pos[2] for pos in smoke_positions]
        plt.plot(times - start_time, smoke_altitudes, color=colors[i], linestyle='--',
                 label=f'烟幕{i + 1}高度', linewidth=2, alpha=0.7)

    plt.xlabel('时间 (s)')
    plt.ylabel('高度 (m)')
    plt.title('高度变化')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # 4. 单独遮蔽时间分布
```

```
    ax4 = plt.subplot(2, 2, 4)
    plt.bar(range(1, 4), individual_cover_times, color=colors, alpha=0.7)
    plt.xlabel('烟幕弹序号')
    plt.ylabel('遮蔽时间 (s)')
    plt.title('各烟幕弹单独遮蔽时间')
    for i, time in enumerate(individual_cover_times):
        plt.text(i + 1, time + 0.1, f'{time:.1f}s', ha='center', va='bottom',
fontsize=12)
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('three_smokes_cover_analysis.png', dpi=300, bbox_inches='tight')
    plt.show()

    # 计算总有效遮蔽时间
    effective_qualities = [q for q in all_qualities if q > 0.1]
    total_effective_time = len(effective_qualities) * (times[1] - times[0])

    print(f"\n 总有效遮蔽时间 (>0.1 质量阈值): {total_effective_time:.3f}s")
    return total_effective_time


if __name__ == "__main__":
    best_params, best_value, individual_cover_times = efficient_optimization()

    v_u, theta_deg = best_params[0], best_params[1]
    smoke_params = best_params[2:]
    total_effective_time = validate_and_visualize_all_smokes(v_u, theta_deg,
smoke_params, individual_cover_times)

    print(f"无人机速度: {v_u:.6f} m/s")
    print(f"无人机航向: {theta_deg:.6f}° ")
    print(f"总遮蔽积分: {best_value:.6f}")
    print(f"总有效遮蔽时间: {total_effective_time:.3f}s")
    print(f"各烟幕弹单独遮蔽时间: {individual_cover_times}")
```

附录 4 问题四

介绍：该代码是用 python 写的

```
# ## 烟幕干扰弹投放策略优化 - 问题 4 (差分进化算法)
# 差分进化，t4
import numpy as np
```

```
from scipy.optimize import minimize, Bounds
import matplotlib.pyplot as plt
import pandas as pd
from datetime import datetime
import time
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['font.family'] = 'sans-serif'
plt.rcParams['text.usetex'] = False
g = 9.8
smoke_radius = 10
smoke_sink_speed = 3
smoke_effective_duration = 20
missile_speed = 300
FY1_initial = np.array([17800, 0, 1800])
FY2_initial = np.array([12000, 1400, 1400])
FY3_initial = np.array([6000, -3000, 700])
M1_initial = np.array([20000, 0, 2000])
# 目标点
target_point = np.array([0, 200, 0])
fake_target = np.array([0, 0, 0])
def calculate_missile_direction_m1():
    direction = fake_target - M1_initial
    return direction / np.linalg.norm(direction)
missile_direction_m1 = calculate_missile_direction_m1()
def missile_position_m1(t):
    return M1_initial + missile_speed * t * missile_direction_m1
def calculate_smoke_burst_single(uav_initial, params):
    v_u, theta_deg, t1, t2 = params
    theta = np.deg2rad(theta_deg)
    uav_velocity_vector = v_u * np.array([np.cos(theta), np.sin(theta), 0])
    drop_position = uav_initial + uav_velocity_vector * t1
    horizontal_displacement = uav_velocity_vector * t2
    vertical_displacement = 0.5 * g * t2 ** 2
    burst_position = drop_position + horizontal_displacement - np.array([0, 0,
vertical_displacement])
    burst_time = t1 + t2
    return burst_position, burst_time, drop_position
def calculate_cover_quality_improved(M, S, P):
    try:
        MS = S - M
        MP = P - M
        distance_MS = np.linalg.norm(MS)
        norm_MP = np.linalg.norm(MP)
```

```
            if distance_MS < 1e-12 or norm_MP < 1e-12:
                return 0.0
            cos_beta = np.dot(MP, MS) / (norm_MP * distance_MS)
            critical_angle = np.arcsin(min(1.0, smoke_radius / distance_MS))
            cos_critical = np.cos(critical_angle)
            if cos_beta >= cos_critical:
                relative_position = (cos_beta - cos_critical) / (1 - cos_critical + 1e-
10)
                quality = np.sqrt(relative_position)
            else:
                falloff = 1.0 - (cos_critical - cos_beta) / (cos_critical + 1.0)
                quality = max(0.0, 0.2 * falloff ** 3)
            return min(1.0, max(0.0, quality))
        except:
            return 0.0
def calculate_total_cover_integral_multi(params_list, uav_initials, debug=False):
        num_smokes = len(params_list)
        burst_positions = []
        burst_times = []
        for i in range(num_smokes):
            params = params_list[i]
            uav_initial = uav_initials[i]
            v_u, theta_deg, t1, t2 = params
            if not (70 <= v_u <= 140 and 0 <= theta_deg <= 360 and 0 <= t1 <= 10 and
0 <= t2 <= 20):
                if                          hasattr(calculate_total_cover_integral_multi,
'return_single_contributions'):
                    return 0.0, [0.0] * num_smokes
                return 0.0
            if t1 + t2 > 30:
                if                          hasattr(calculate_total_cover_integral_multi,
'return_single_contributions'):
                    return 0.0, [0.0] * num_smokes
                return 0.0
            burst_pos,  burst_time,  _  =  calculate_smoke_burst_single(uav_initial,
params)
            if burst_pos[2] < 0:
                if                          hasattr(calculate_total_cover_integral_multi,
'return_single_contributions'):
                    return 0.0, [0.0] * num_smokes
                return 0.0
            burst_positions.append(burst_pos)
            burst_times.append(burst_time)
        total_integral = 0.0
```

```
    dt = 0.01
    t_start_sim = 0.0
    t_end_sim = 70.0
    current_t = t_start_sim
    while current_t < t_end_sim:
        M_t = missile_position_m1(current_t)
        max_quality = 0.0
        for i in range(num_smokes):
            burst_time = burst_times[i]
            if current_t < burst_time or (current_t - burst_time) > smoke_effective_duration:
                continue
            sink_distance = smoke_sink_speed * (current_t - burst_time)
            S_t = burst_positions[i] - np.array([0, 0, sink_distance])
            if S_t[2] < 0:
                continue
            quality = calculate_cover_quality_improved(M_t, S_t, target_point)
            if quality > max_quality:
                max_quality = quality
        total_integral += max_quality * dt
        current_t += dt
    if debug:
        print(f"调试信息 - 总遮蔽积分: {total_integral:.6f}")
    if hasattr(calculate_total_cover_integral_multi, 'return_single_contributions'):
        single_contributions = []
        for i in range(len(params_list)):

single_contributions.append(calculate_single_uav_cover(uav_initials[i], params_list[i]))
        return total_integral, single_contributions
    return total_integral
def calculate_single_uav_cover(uav_initial, params, debug=False):
    v_u, theta_deg, t1, t2 = params
    if not (70 <= v_u <= 140 and 0 <= theta_deg <= 360 and 0 <= t1 <= 10 and 0 <= t2 <= 20):
        return 0.0
    if t1 + t2 > 30:
        return 0.0
    burst_pos, burst_time, _ = calculate_smoke_burst_single(uav_initial, params)
    if burst_pos[2] < 0:
        return 0.0
    total_integral = 0.0
    dt = 0.01
    t_start_sim = 0.0
```

```
    t_end_sim = 80.0
    current_t = t_start_sim
    while current_t < t_end_sim:
        M_t = missile_position_m1(current_t)
        if current_t < burst_time or (current_t - burst_time) > smoke_effective_duration:
            current_t += dt
            continue
        sink_distance = smoke_sink_speed * (current_t - burst_time)
        S_t = burst_pos - np.array([0, 0, sink_distance])
        if S_t[2] < 0:
            current_t += dt
            continue
        quality = calculate_cover_quality_improved(M_t, S_t, target_point)
        total_integral += quality * dt
        current_t += dt
    if debug:
        print(f"单无人机遮蔽积分: {total_integral:.6f}")
    return total_integral
def fitness_function_multi(params, debug=False):
    params_fy1 = params[0:4]
    params_fy2 = params[4:8]
    params_fy3 = params[8:12]
    params_list = [params_fy1, params_fy2, params_fy3]
    uav_initials = [FY1_initial, FY2_initial, FY3_initial]
    total_integral = calculate_total_cover_integral_multi(params_list, uav_initials, debug)
    return -total_integral
POPULATION_SIZE_DE = 50
MAX_GENERATIONS_DE = 1000
F_DE = 0.8
CR_DE = 0.9
PARAM_BOUNDS_DE = [
    (70, 140), (0, 360), (0.1, 10), (0.1, 20),
    (70, 140), (0, 360), (0.1, 10), (0.1, 20),
    (70, 140), (0, 360), (0.1, 10), (0.1, 20)
]
NUM_PARAMS = len(PARAM_BOUNDS_DE)
def initialize_population_de(pop_size, bounds):
    population = np.zeros((pop_size, NUM_PARAMS))
    for i in range(NUM_PARAMS):
        min_val, max_val = bounds[i]
        population[:, i] = np.random.uniform(min_val, max_val, size=pop_size)
    return population
```

```
def repair_individual(individual, bounds):
    repaired = np.clip(individual, [b[0] for b in bounds], [b[1] for b in bounds])
    return repaired
def differential_evolution_optimizer(fitness_func, bounds, pop_size, generations, F, CR):
    population = initialize_population_de(pop_size, bounds)
    fitness = np.array([fitness_func(ind) for ind in population])
    best_fitness_overall = np.min(fitness)
    best_individual_overall = population[np.argmin(fitness)].copy()
    best_fitness_history = [best_fitness_overall]
    avg_fitness_history = [np.mean(fitness)]
    print(f"差分进化 - 初始最佳适应度（负遮蔽时间）: {best_fitness_overall:.6f}")
    print(f"差分进化 - 初始平均适应度（负遮蔽时间）: {np.mean(fitness):.6f}")
    for generation in range(generations):
        new_population = np.copy(population)
        for i in range(pop_size):
            candidates = [j for j in range(pop_size) if j != i]
            r1, r2, r3 = np.random.choice(candidates, 3, replace=False)
            mutant_vector = population[r1] + F * (population[r2] - population[r3])
            cross_points = np.random.rand(NUM_PARAMS) < CR
            if not np.any(cross_points):
                cross_points[np.random.randint(0, NUM_PARAMS)] = True
            trial_vector = np.where(cross_points, mutant_vector, population[i])
            repaired_trial = repair_individual(trial_vector, bounds)
            trial_fitness = fitness_func(repaired_trial)
            if trial_fitness < fitness[i]:
                new_population[i] = repaired_trial
                fitness[i] = trial_fitness
                if trial_fitness < best_fitness_overall:
                    best_fitness_overall = trial_fitness
                    best_individual_overall = repaired_trial.copy()
        population = new_population
        best_fitness_history.append(best_fitness_overall)
        avg_fitness_history.append(np.mean(fitness))
        if (generation + 1) % 10 == 0:
            print(f"差分进化 - 代 {generation + 1}/{generations}, 当前最佳适应度: {best_fitness_overall:.6f}, 平均: {np.mean(fitness):.6f}")
    print(f"\n--- 差分进化优化完成 ---")
    print(f"找到的最优适应度: {best_fitness_overall:.6f}")
    return best_individual_overall, best_fitness_history, avg_fitness_history
def local_refinement(best_params):
    print("\n--- 进行局部精细化优化 ---")
```

```
    bounds_list = PARAM_BOUNDS_DE
    lower_bounds = [b[0] for b in bounds_list]
    upper_bounds = [b[1] for b in bounds_list]
    bounds_obj = Bounds(lower_bounds, upper_bounds)
    try:
        result = minimize(
            fitness_function_multi,
            best_params,
            method='Nelder-Mead',
            bounds=bounds_obj,
            options={'maxiter': 800, 'disp': True}
        )
        if result.success:
            refined_fitness = -result.fun
            print(f"局部精细化成功！精细化后最大遮蔽时间：
{refined_fitness:.6f} s")
            return result.x, refined_fitness
        else:
            print(f"局部精细化未能成功: {result.message}")
            original_fitness = -fitness_function_multi(best_params)
            return best_params, original_fitness
    except Exception as e:
        print(f"局部精细化错误: {e}")
        original_fitness = -fitness_function_multi(best_params)
        return best_params, original_fitness
def validate_and_visualize_multi(best_params_refined):
    params_fy1 = best_params_refined[0:4]
    params_fy2 = best_params_refined[4:8]
    params_fy3 = best_params_refined[8:12]
    params_list = [params_fy1, params_fy2, params_fy3]
    uav_initials = [FY1_initial, FY2_initial, FY3_initial]
    burst_pos1,          burst_time1,          drop_pos1          =
calculate_smoke_burst_single(FY1_initial, params_fy1)
    burst_pos2,          burst_time2,          drop_pos2          =
calculate_smoke_burst_single(FY2_initial, params_fy2)
    burst_pos3,          burst_time3,          drop_pos3          =
calculate_smoke_burst_single(FY3_initial, params_fy3)
    print(f"\n=== 问题 4：最优投放策略总结 (3 无人机干扰 M1) ===")
    print("\n--- 无人机 FY1 策略 ---")
    print(f"  飞行速度 v: {params_fy1[0]:.3f} m/s")
    print(f"  航向角 θ: {params_fy1[1]:.3f}° ")
    print(f"  投放时间 t1: {params_fy1[2]:.3f} s")
    print(f"  起爆延迟 t2: {params_fy1[3]:.3f} s")
    print(f"  投放位置: ({drop_pos1[0]:.1f}, {drop_pos1[1]:.1f},
```

```
{drop_pos1[2]:.1f}) m")
    print(f"  烟幕爆开时间: {burst_time1:.3f} s")
    print(f"  烟 幕 爆 开 位 置 : ({burst_pos1[0]:.1f}, {burst_pos1[1]:.1f},
{burst_pos1[2]:.1f}) m")
    print("\n--- 无人机 FY2 策略 ---")
    print(f"  飞行速度 v: {params_fy2[0]:.3f} m/s")
    print(f"  航向角 θ: {params_fy2[1]:.3f}° ")
    print(f"  投放时间 t1: {params_fy2[2]:.3f} s")
    print(f"  起爆延迟 t2: {params_fy2[3]:.3f} s")
    print(f"  投 放 位 置 : ({drop_pos2[0]:.1f}, {drop_pos2[1]:.1f},
{drop_pos2[2]:.1f}) m")
    print(f"  烟幕爆开时间: {burst_time2:.3f} s")
    print(f"  烟 幕 爆 开 位 置 : ({burst_pos2[0]:.1f}, {burst_pos2[1]:.1f},
{burst_pos2[2]:.1f}) m")
    print("\n--- 无人机 FY3 策略 ---")
    print(f"  飞行速度 v: {params_fy3[0]:.3f} m/s")
    print(f"  航向角 θ: {params_fy3[1]:.3f}° ")
    print(f"  投放时间 t1: {params_fy3[2]:.3f} s")
    print(f"  起爆延迟 t2: {params_fy3[3]:.3f} s")
    print(f"  投 放 位 置 : ({drop_pos3[0]:.1f}, {drop_pos3[1]:.1f},
{drop_pos3[2]:.1f}) m")
    print(f"  烟幕爆开时间: {burst_time3:.3f} s")
    print(f"  烟 幕 爆 开 位 置 : ({burst_pos3[0]:.1f}, {burst_pos3[1]:.1f},
{burst_pos3[2]:.1f}) m")
    total_integral = calculate_total_cover_integral_multi(params_list, uav_initials,
debug=True)
    calculate_total_cover_integral_multi.return_single_contributions = True
    total_integral,                    single_contributions                    =
calculate_total_cover_integral_multi(params_list, uav_initials, debug=True)
    calculate_total_cover_integral_multi.return_single_contributions = False
    fy1_int, fy2_int, fy3_int = single_contributions
    print(f"\n--- 单无人机遮蔽贡献 ---")
    print(f"  FY1 单独遮蔽时间: {fy1_int:.6f} s")
    print(f"  FY2 单独遮蔽时间: {fy2_int:.6f} s")
    print(f"  FY3 单独遮蔽时间: {fy3_int:.6f} s")
    return total_integral, single_contributions
def                                save_results_to_excel(best_params_refined,
total_integral,single_contributions):
    params_fy1 = best_params_refined[0:4]
    params_fy2 = best_params_refined[4:8]
    params_fy3 = best_params_refined[8:12]
    burst_pos1,          burst_time1,          drop_pos1          =
calculate_smoke_burst_single(FY1_initial, params_fy1)
    burst_pos2,          burst_time2,          drop_pos2          =
```

```
calculate_smoke_burst_single(FY2_initial, params_fy2)
    burst_pos3, burst_time3, drop_pos3 = calculate_smoke_burst_single(FY3_initial, params_fy3)
    results_data = {
        '无人机': ['FY1', 'FY2', 'FY3', '总计'],
        '飞行速度 v_u (m/s)': [f"{params_fy1[0]:.3f}", f"{params_fy2[0]:.3f}", f"{params_fy3[0]:.3f}", '-'],
        '航向角 theta (°)': [f"{params_fy1[1]:.3f}", f"{params_fy2[1]:.3f}", f"{params_fy3[1]:.3f}", '-'],
        '投放时间 t1 (s)': [f"{params_fy1[2]:.3f}", f"{params_fy2[2]:.3f}", f"{params_fy3[2]:.3f}", '-'],
        '起爆延迟 t2 (s)': [f"{params_fy1[3]:.3f}", f"{params_fy2[3]:.3f}", f"{params_fy3[3]:.3f}", '-'],
        '投放 X 坐标 (m)': [f"{drop_pos1[0]:.1f}", f"{drop_pos2[0]:.1f}", f"{drop_pos3[0]:.1f}", '-'],
        '投放 Y 坐标 (m)': [f"{drop_pos1[1]:.1f}", f"{drop_pos2[1]:.1f}", f"{drop_pos3[1]:.1f}", '-'],
        '投放 Z 坐标 (m)': [f"{drop_pos1[2]:.1f}", f"{drop_pos2[2]:.1f}", f"{drop_pos3[2]:.1f}", '-'],
        '烟幕爆开时间 (s)': [f"{burst_time1:.3f}", f"{burst_time2:.3f}", f"{burst_time3:.3f}", '-'],
        '烟幕爆开 X 坐标(m)': [f"{burst_pos1[0]:.1f}", f"{burst_pos2[0]:.1f}", f"{burst_pos3[0]:.1f}", '-'],
        '烟幕爆开 Y 坐标(m)': [f"{burst_pos1[1]:.1f}", f"{burst_pos2[1]:.1f}", f"{burst_pos3[1]:.1f}", '-'],
        '烟幕爆开 Z 坐标(m)': [f"{burst_pos1[2]:.1f}", f"{burst_pos2[2]:.1f}", f"{burst_pos3[2]:.1f}", '-'],
        '单独遮蔽贡献 (s)': [f"{single_contributions[0]:.6f}", f"{single_contributions[1]:.6f}",
                              f"{single_contributions[2]:.6f}", '-'],
        '总遮蔽时间(s)': ['-', '-', '-', f"{total_integral:.6f}"]
    }
    results_df = pd.DataFrame(results_data)
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f'result2.xlsx'  # 按题目要求保存为 result2.xlsx
    results_df.to_excel(filename, index=False)
    print(f"\n 结果已保存到: {filename}")
    return results_df
def plot_fitness_history(best_history, avg_history):
    plt.figure(figsize=(10, 6))
    plt.plot([-f for f in best_history], 'b-', label='最佳遮蔽时间')
    plt.plot([-f for f in avg_history], 'r-', label='平均遮蔽时间')
    plt.xlabel('代数')
    plt.ylabel('遮蔽时间 (s)')
```

```
    plt.title('差分进化优化过程')
    plt.legend()
    plt.grid(True)
    plt.savefig('problem4_DE_evolution.png', dpi=300)
    print("进化曲线图已保存为 'problem4_DE_evolution.png'")
def plot_multi_visualizations(best_params_refined, single_contributions, best_history, avg_history):
    params_fy1 = best_params_refined[0:4]
    params_fy2 = best_params_refined[4:8]
    params_fy3 = best_params_refined[8:12]
    params_list = [params_fy1, params_fy2, params_fy3]
    uav_initials = [FY1_initial, FY2_initial, FY3_initial]
    plt.figure(figsize=(10, 6))
    plt.plot([-f for f in best_history], 'b-', label='最佳遮蔽时间')
    plt.plot([-f for f in avg_history], 'r-', label='平均遮蔽时间')
    plt.xlabel('迭代代数')
    plt.ylabel('遮蔽时间 (s)')
    plt.title('DE 算法适应度进化曲线')
    plt.legend()
    plt.grid(True)
    plt.savefig('de_evolution.png', dpi=300)
    plt.close()
    plt.figure(figsize=(8, 6))
    uav_names = ['FY1', 'FY2', 'FY3']
    bars = plt.bar(uav_names, single_contributions, color=['blue', 'green', 'red'])
    plt.xlabel('无人机名称')
    plt.ylabel('遮蔽时间 (s)')
    plt.title('单无人机有效干扰时长对比')
    for bar, value in zip(bars, single_contributions):
        plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.1,
                 f'{value:.2f}', ha='center', va='bottom')
    plt.grid(True, axis='y')
    plt.savefig('single_uav_contribution.png', dpi=300)
    plt.close()
    plt.figure(figsize=(12, 6))
    time_points = np.linspace(0, 80, 800)
    for i, (params, uav_initial) in enumerate(zip(params_list, uav_initials)):
        burst_pos, burst_time, _ = calculate_smoke_burst_single(uav_initial, params)
        quality_over_time = []
        for t in time_points:
            if t < burst_time or (t - burst_time) > smoke_effective_duration:
                quality_over_time.append(0)
                continue
```

```
                M_t = missile_position_m1(t)
                sink_distance = smoke_sink_speed * (t - burst_time)
                S_t = burst_pos - np.array([0, 0, sink_distance])
                if S_t[2] < 0:
                    quality_over_time.append(0)
                    continue
                quality = calculate_cover_quality_improved(M_t, S_t, target_point)
                quality_over_time.append(quality)
            plt.plot(time_points, quality_over_time, label=f'FY{i + 1}', linewidth=2)
    plt.xlabel('时间 (s)')
    plt.ylabel('遮蔽质量')
    plt.title('遮蔽质量随时间动态变化')
    plt.legend()
    plt.grid(True)
    plt.ylim(0, 1.1)
    plt.savefig('cover_quality_dynamic.png', dpi=300)
    plt.close()
    plt.figure(figsize=(10, 8))
    missile_traj_x = [M1_initial[0], fake_target[0]]
    missile_traj_y = [M1_initial[1], fake_target[1]]
    plt.plot(missile_traj_x, missile_traj_y, 'r-', linewidth=2, label='导弹轨迹')
    colors = ['orange', 'green', 'purple']
    labels = ['FY1 烟幕', 'FY2 烟幕', 'FY3 烟幕']
    for i, (params, uav_initial) in enumerate(zip(params_list, uav_initials)):
        burst_pos, _, _ = calculate_smoke_burst_single(uav_initial, params)
        plt.scatter(burst_pos[0], burst_pos[1], color=colors[i], s=100,
label=labels[i], marker='o')
    plt.scatter(target_point[0], target_point[1], color='blue', s=150, label='真目标',
marker='s')
    plt.scatter(fake_target[0], fake_target[1], color='black', s=150, label='假目标',
marker='^')
    plt.xlabel('X 坐标 (m)')
    plt.ylabel('Y 坐标 (m)')
    plt.title('导弹与烟幕空间轨迹（XY 平面）')
    plt.legend()
    plt.grid(True)
    plt.axis('equal')
    plt.savefig('spatial_trajectory.png', dpi=300)
    plt.close()
    plt.figure(figsize=(10, 6))
    all_qualities = []
    dt = 0.1
    current_t = 0
    while current_t < 80:
```

```
            M_t = missile_position_m1(current_t)
            max_quality = 0.0
            for i, (params, uav_initial) in enumerate(zip(params_list, uav_initials)):
                burst_pos, burst_time, _ = calculate_smoke_burst_single(uav_initial,
params)
                if current_t < burst_time or (current_t - burst_time) >
smoke_effective_duration:
                    continue
                sink_distance = smoke_sink_speed * (current_t - burst_time)
                S_t = burst_pos - np.array([0, 0, sink_distance])
                if S_t[2] < 0:
                    continue
                quality = calculate_cover_quality_improved(M_t, S_t, target_point)
                if quality > max_quality:
                    max_quality = quality
            all_qualities.append(max_quality)
            current_t += dt
        bins = [0, 0.2, 0.4, 0.6, 0.8, 1.0]
        hist, bin_edges = np.histogram(all_qualities, bins=bins)
        bin_centers = [(bin_edges[i] + bin_edges[i + 1]) / 2 for i in range(len(bin_edges)
- 1)]
        bin_labels = [f'{bin_edges[i]:.1f}-{bin_edges[i + 1]:.1f}' for i in
range(len(bin_edges) - 1)]
        plt.bar(bin_labels, hist / sum(hist) * 100, alpha=0.7)
        plt.xlabel('遮蔽质量区间')
        plt.ylabel('时间占比 (%)')
        plt.title('遮蔽质量分布直方图')
        plt.grid(True, axis='y')
        for i, v in enumerate(hist / sum(hist) * 100):
            plt.text(i, v + 1, f'{v:.1f}%', ha='center')
        plt.savefig('cover_quality_distribution.png', dpi=300)
        plt.close()
        print("所有可视化图表已保存完成!")

if __name__ == "__main__":
        print("=== 问题 4：无人机协同烟幕投放策略优化 ===")
        print(f"开始时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
        print("\n[STEP 1/4] 运行差分进化算法...")
        start_time_de = time.time()
        best_params_de, best_history, avg_history = differential_evolution_optimizer(
            fitness_function_multi,
            PARAM_BOUNDS_DE,
            POPULATION_SIZE_DE,
            MAX_GENERATIONS_DE,
```

```
            F_DE,
            CR_DE
        )
        end_time_de = time.time()
        print(f"差分进化耗时: {end_time_de - start_time_de:.2f} 秒")
        print("\n[STEP 2/4] 进行局部精细化...")
        start_time_refine = time.time()
        refined_params, refined_fitness = local_refinement(best_params_de)
        end_time_refine = time.time()
        print(f"局部精细化耗时: {end_time_refine - start_time_refine:.2f} 秒")
        print("\n[STEP 3/4] 验证并输出最终策略...")
        final_total_integral, single_contributions = validate_and_visualize_multi(refined_params)
        print("\n[STEP 4/4] 保存结果到 Excel 并生成可视化...")
        save_results_to_excel(refined_params, final_total_integral, single_contributions)
        plot_multi_visualizations(refined_params, single_contributions, best_history, avg_history)
        print("\n=== 优化计算完成 ===")
        print(f"结束时间: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
```

附录 3 问题五

介绍：该代码是用 python 语言写的

```
import numpy as np
from scipy.optimize import minimize, differential_evolution
import matplotlib.pyplot as plt
import pandas as pd
from datetime import datetime
import numba
from mpl_toolkits.mplot3d import Axes3D

plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
plt.rcParams["axes.unicode_minus"] = False

g = 9.8
smoke_radius = 10
smoke_sink_speed = 3
missile_speed = 300

target_point = np.array([0, 200, 0])
fake_target = np.array([0, 0, 0])

missiles_initial = np.array([
```

```
    [20000, 0, 2000],    # M1
    [19000, 600, 2100],    # M2
    [18000, -600, 1900]    # M3
])

uavs_initial = np.array([
    [17800, 0, 1800],    # FY1
    [12000, 1400, 1400],    # FY2
    [6000, -3000, 700],    # FY3
    [11000, 2000, 1800],    # FY4
    [13000, -2000, 1300]    # FY5
])

missile_directions = np.zeros((3, 3))
for i in range(3):
    direction = fake_target - missiles_initial[i]
    missile_directions[i] = direction / np.linalg.norm(direction)


@numba.njit(fastmath=True)
def calculate_missile_position_numba(missile_idx, t):
    return missiles_initial[missile_idx] + missile_speed * t *
missile_directions[missile_idx]


@numba.njit(fastmath=True)
def calculate_cover_quality_numba(M, S, P, smoke_radius):
    try:
        MS = S - M
        MP = P - M
        distance_MS = np.linalg.norm(MS)

        if distance_MS < smoke_radius:
            return 1.0 - (distance_MS / smoke_radius) ** 2

        distance_MP = np.linalg.norm(MP)
        if distance_MS < 1e-12 or distance_MP < 1e-12:
            return 0.0

        cos_beta = np.dot(MP, MS) / (distance_MP * distance_MS)
        critical_angle = np.arcsin(min(1.0, smoke_radius / distance_MS))
        cos_critical = np.cos(critical_angle)

        if cos_beta >= cos_critical:
```

```
                relative_position = (cos_beta - cos_critical) / (1 - cos_critical + 1e-
10)
                quality = np.sqrt(relative_position)
            else:
                falloff = 1.0 - (cos_critical - cos_beta) / (cos_critical + 1.0)
                quality = max(0.0, 0.2 * falloff ** 3)

            return min(1.0, max(0.0, quality))
        except:
            return 0.0


def calculate_smoke_burst(uav_idx, params):
    v_u, theta_deg, t1, t2 = params
    theta = np.deg2rad(theta_deg)
    uav_velocity = v_u * np.array([np.cos(theta), np.sin(theta), 0])

    drop_position = uavs_initial[uav_idx] + uav_velocity * t1
    fall_time = t2
    horizontal_displacement = uav_velocity * fall_time
    vertical_displacement = 0.5 * g * fall_time ** 2
    burst_position = drop_position + horizontal_displacement - np.array([0, 0,
vertical_displacement])
    burst_time = t1 + fall_time

    return burst_position, burst_time


def objective_function_multi(params, uav_smoke_assignments):
    total_integral = 0
    param_idx = 0
    smoke_info = []

    for uav_idx, num_smokes in enumerate(uav_smoke_assignments):
        for _ in range(num_smokes):
            v_u = params[param_idx]
            theta_deg = params[param_idx + 1]
            t1 = params[param_idx + 2]
            t2 = params[param_idx + 3]
            param_idx += 4

            burst_pos, burst_time = calculate_smoke_burst(uav_idx, [v_u,
theta_deg, t1, t2])
            end_time = burst_time + 20
```

```
                smoke_info.append((uav_idx, burst_pos, burst_time, end_time))

    if not smoke_info:
        return 0.0

    all_start_times = [info[2] for info in smoke_info]
    all_end_times = [info[3] for info in smoke_info]
    start_time = max(0, min(all_start_times) - 1)
    end_time = max(all_end_times) + 1
    time_steps = np.arange(start_time, end_time, 0.5)

    for t in time_steps:
        max_quality = np.zeros(3)

        for (uav_idx, burst_pos, burst_time, end_time) in smoke_info:
            if t < burst_time or t > end_time:
                continue

            sink_distance = smoke_sink_speed * (t - burst_time)
            S_t = burst_pos - np.array([0, 0, sink_distance])
            if S_t[2] < 0:
                continue

            for missile_idx in range(3):
                M_t = calculate_missile_position_numba(missile_idx, t)
                quality = calculate_cover_quality_numba(M_t, S_t,
target_point, smoke_radius)
                if quality > max_quality[missile_idx]:
                    max_quality[missile_idx] = quality

        total_integral += np.sum(max_quality) * 0.5

    return -total_integral


def staged_optimization(uav_smoke_assignments):
    """分阶段优化：先优化投放时间，再优化所有参数"""
    print("=== 第一阶段：优化投放和起爆时间 ===")

    # 第一阶段：只优化投放和起爆时间
    bounds_stage1 = []
    initial_guess_stage1 = []

    for uav_idx, num_smokes in enumerate(uav_smoke_assignments):
```

```
            for _ in range(num_smokes):
                # 只优化 t1 和 t2，固定速度和航向
                bounds_stage1.extend([(0, 10), (0, 20)])
                initial_guess_stage1.extend([2.0, 6.0])

    # 使用固定速度和航向
    fixed_params = []
    for uav_idx, num_smokes in enumerate(uav_smoke_assignments):
        for _ in range(num_smokes):
            # 为每架无人机设置合理的初始速度和航向
            if uav_idx == 0:  # FY1
                fixed_params.extend([120, 180])
            elif uav_idx == 1:  # FY2
                fixed_params.extend([110, 190])
            elif uav_idx == 2:  # FY3
                fixed_params.extend([130, 170])
            elif uav_idx == 3:  # FY4
                fixed_params.extend([125, 200])
            else:  # FY5
                fixed_params.extend([115, 160])

    def stage1_objective(time_params):
        full_params = []
        time_idx = 0
        param_idx = 0
        for uav_idx, num_smokes in enumerate(uav_smoke_assignments):
            for _ in range(num_smokes):
                full_params.extend([fixed_params[param_idx],
fixed_params[param_idx + 1],
                                    time_params[time_idx],
time_params[time_idx + 1]])
                time_idx += 2
                param_idx += 2
        return objective_function_multi(full_params, uav_smoke_assignments)

    # 第一阶段优化
    result_stage1 = differential_evolution(
        stage1_objective,
        bounds_stage1,
        maxiter=50,
        popsize=10,
        mutation=(0.7, 1.0),
        recombination=0.8,
        tol=0.1,
```

```
        disp=True
    )

    print(f"第一阶段完成，最优值: {-result_stage1.fun:.2f}")

    # 第二阶段：使用第一阶段结果作为初始值，优化所有参数
    print("\n=== 第二阶段：优化所有参数 ===")

    # 构建第二阶段的初始猜测
    initial_guess_stage2 = []
    time_idx = 0
    param_idx = 0
    for uav_idx, num_smokes in enumerate(uav_smoke_assignments):
        for _ in range(num_smokes):
            initial_guess_stage2.extend([
                fixed_params[param_idx],
                fixed_params[param_idx + 1],
                result_stage1.x[time_idx],
                result_stage1.x[time_idx + 1]
            ])
            time_idx += 2
            param_idx += 2

    # 完整参数边界
    bounds_full = []
    for _ in range(sum(uav_smoke_assignments)):
        bounds_full.extend([(70, 140), (0, 360), (0, 10), (0, 20)])

    # 第二阶段优化
    result_full = differential_evolution(
        lambda x: objective_function_multi(x, uav_smoke_assignments),
        bounds_full,
        x0=initial_guess_stage2,
        maxiter=100,
        popsize=12,
        mutation=(0.5, 1.0),
        recombination=0.7,
        tol=0.05,
        disp=True,
        polish=True,
        strategy='best1bin'
    )

    return result_full
```

```
def print_detailed_results(result, uav_smoke_assignments):
    """详细输出所有参数"""
    print("\n" + "=" * 80)
    print("优化结果详细参数")
    print("=" * 80)

    param_idx = 0
    for uav_idx, num_smokes in enumerate(uav_smoke_assignments):
        uav_name = f'FY{uav_idx + 1}'
        print(f"\n{uav_name} 投放策略:")
        print("-" * 50)

        for smoke_idx in range(num_smokes):
            v_u = result.x[param_idx]
            theta_deg = result.x[param_idx + 1]
            t1 = result.x[param_idx + 2]
            t2 = result.x[param_idx + 3]

            burst_pos, burst_time = calculate_smoke_burst(uav_idx, [v_u,
theta_deg, t1, t2])

            print(f"烟幕弹 {smoke_idx + 1}:")
            print(f"  飞行速度: {v_u:.2f} m/s")
            print(f"  航向角: {theta_deg:.2f}°")
            print(f"  投放时间: {t1:.2f} s")
            print(f"  起爆延迟: {t2:.2f} s")
            print(f"  爆开时间: {burst_time:.2f} s")
            print(f"  爆开位置: ({burst_pos[0]:.2f}, {burst_pos[1]:.2f},
{burst_pos[2]:.2f}) m")
            print()

            param_idx += 4


def visualize_scenario(result, uav_smoke_assignments):
    """可视化无人机、导弹和烟幕位置"""
    fig = plt.figure(figsize=(15, 10))

    ax1 = fig.add_subplot(121, projection='3d')

    uav_colors = ['red', 'blue', 'green', 'orange', 'purple']
    uav_names = ['FY1', 'FY2', 'FY3', 'FY4', 'FY5']
```

```
    for i, (uav_pos, color, name) in enumerate(zip(uavs_initial, uav_colors, uav_names)):
        ax1.scatter(uav_pos[0], uav_pos[1], uav_pos[2], c=color, s=100, label=name, marker='^')

    missile_colors = ['darkred', 'darkblue', 'darkgreen']
    missile_names = ['M1', 'M2', 'M3']

    for i, (missile_pos, color, name) in enumerate(zip(missiles_initial, missile_colors, missile_names)):
        ax1.scatter(missile_pos[0], missile_pos[1], missile_pos[2], c=color, s=100, label=name, marker='o')

        trajectory = np.array([missile_pos, fake_target])
        ax1.plot(trajectory[:, 0], trajectory[:, 1], trajectory[:, 2],
                 c=color, linestyle='--', alpha=0.7)

    ax1.scatter(target_point[0], target_point[1], target_point[2],
                c='gold', s=200, label='真目标', marker='*')
    ax1.scatter(fake_target[0], fake_target[1], fake_target[2],
                c='black', s=200, label='假目标', marker='x')

    param_idx = 0
    smoke_positions = []
    for uav_idx, num_smokes in enumerate(uav_smoke_assignments):
        for _ in range(num_smokes):
            v_u = result.x[param_idx]
            theta_deg = result.x[param_idx + 1]
            t1 = result.x[param_idx + 2]
            t2 = result.x[param_idx + 3]

            burst_pos, _ = calculate_smoke_burst(uav_idx, [v_u, theta_deg, t1, t2])
            smoke_positions.append(burst_pos)
            ax1.scatter(burst_pos[0], burst_pos[1], burst_pos[2],
                        c=uav_colors[uav_idx], s=80, marker='s', alpha=0.7)

            param_idx += 4

    ax1.set_xlabel('X 坐标 (m)')
    ax1.set_ylabel('Y 坐标 (m)')
    ax1.set_zlabel('Z 坐标 (m)')
    ax1.set_title('3D 场景可视化')
```

```
    ax1.legend(bbox_to_anchor=(1.05, 1), loc='upper left')

    ax2 = fig.add_subplot(122)

    for i, (uav_pos, color, name) in enumerate(zip(uavs_initial, uav_colors, uav_names)):
        ax2.scatter(uav_pos[0], uav_pos[1], c=color, s=100, label=name, marker='^')

    for i, (missile_pos, color, name) in enumerate(zip(missiles_initial, missile_colors, missile_names)):
        ax2.scatter(missile_pos[0], missile_pos[1], c=color, s=100, label=name, marker='o')
        trajectory = np.array([missile_pos, fake_target])
        ax2.plot(trajectory[:, 0], trajectory[:, 1], c=color, linestyle='--', alpha=0.7)

    ax2.scatter(target_point[0], target_point[1], c='gold', s=200, label='真目标', marker='*')
    ax2.scatter(fake_target[0], fake_target[1], c='black', s=200, label='假目标', marker='x')

    for i, smoke_pos in enumerate(smoke_positions):
        ax2.scatter(smoke_pos[0], smoke_pos[1], c='magenta', s=60, marker='s', alpha=0.7)

    ax2.set_xlabel('X 坐标 (m)')
    ax2.set_ylabel('Y 坐标 (m)')
    ax2.set_title('2D 平面投影')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('scenario_visualization.png', dpi=300, bbox_inches='tight')
    plt.show()


def save_results_to_excel(result, uav_smoke_assignments, filename):
    data = []
    param_idx = 0

    for uav_idx, num_smokes in enumerate(uav_smoke_assignments):
        uav_name = f'FY{uav_idx + 1}'
        for smoke_idx in range(num_smokes):
            v_u = result.x[param_idx]
```

```
                theta_deg = result.x[param_idx + 1]
                t1 = result.x[param_idx + 2]
                t2 = result.x[param_idx + 3]
                param_idx += 4

                burst_pos, burst_time = calculate_smoke_burst(uav_idx, [v_u,
theta_deg, t1, t2])

                data.append({
                    '无人机编号': uav_name,
                    '烟幕弹编号': smoke_idx + 1,
                    '飞行速度(m/s)': round(v_u, 2),
                    '航向角(度)': round(theta_deg, 2),
                    '投放时间(s)': round(t1, 2),
                    '起爆时间(s)': round(t2, 2),
                    '爆开时间(s)': round(burst_time, 2),
                    '爆开位置 X(m)': round(burst_pos[0], 2),
                    '爆开位置 Y(m)': round(burst_pos[1], 2),
                    '爆开位置 Z(m)': round(burst_pos[2], 2)
                })

    pd.DataFrame(data).to_excel(filename, index=False)
    print(f"结果已保存到 {filename}")


def main():
    uav_smoke_assignments = [2, 2, 2, 2, 2]

    print("=== 多无人机多烟幕弹优化开始（分阶段优化）===")
    start_time = datetime.now()


    result = staged_optimization(uav_smoke_assignments)

    print(f"\n 优化耗时: {(datetime.now() - start_time).total_seconds():.2f}秒")

    if result.success:
        print(f"优化成功！最优遮蔽积分: {-result.fun:.2f}")
        print_detailed_results(result, uav_smoke_assignments)


        visualize_scenario(result, uav_smoke_assignments)
```

```
            save_results_to_excel(result, uav_smoke_assignments, 'result3.xlsx')
        else:
            print("优化失败，尝试减少烟幕弹数量或调整参数")


if __name__ == "__main__":
    main()
```